```
cloudra/tools/core/workspace_mutation.py:208            change_id=change.change_id,
cloudra/tools/core/workspace_mutation.py:209            before_sha256=change.before_sha256,
cloudra/tools/core/workspace_mutation.py:210            after_sha256=change.after_sha256,
cloudra/tools/core/workspace_mutation.py:211            diff=change.diff,
cloudra/tools/core/workspace_mutation.py:212            message="File change written atomically",
cloudra/tools/core/workspace_mutation.py:213        )
cloudra/tools/core/xtb_frames.py:1  """xtb 输出日志文本原语：终止判定 + 能量/优化收敛/频率/热化学提取。
cloudra/tools/core/xtb_frames.py:2
cloudra/tools/core/xtb_frames.py:3  与 orca_frames 对齐的纯函数模块（GH #26）。xtb 主输出全部走 stdout
cloudra/tools/core/xtb_frames.py:4  （calculator.sh 重定向到 ``<输入stem>.out``）；cclib 1.8.1 的 xtb parser
cloudra/tools/core/xtb_frames.py:5  结果错误（opt 能量取到最后 cycle、hess 热化学为荒谬值），故终态判定与
cloudra/tools/core/xtb_frames.py:6  字段提取一律走文本权威判据，xtb 引擎在 log_parser 中整体跳过 cclib；
cloudra/tools/core/xtb_frames.py:7  guard（runtime/monitor.py）与 log_parser 共用本模块，行为与 orca_frames
cloudra/tools/core/xtb_frames.py:8  同款：
cloudra/tools/core/xtb_frames.py:9
cloudra/tools/core/xtb_frames.py:10 - 正常终止标记 ``normal termination of xtb``（引擎自报）；
cloudra/tools/core/xtb_frames.py:11 - 优化收敛是独立于终止标记的第二判据（GH #21 同款语义）：未收敛的优化
cloudra/tools/core/xtb_frames.py:12   仍打印 normal termination，以 ``GEOMETRY OPTIMIZATION CONVERGED`` /
cloudra/tools/core/xtb_frames.py:13   ``FAILED TO CONVERGE GEOMETRY OPTIMIZATION`` 文本标记为准；
cloudra/tools/core/xtb_frames.py:14 - 能量按文本位置取最后出现的能量行（GH #22 同款语义）：最终
cloudra/tools/core/xtb_frames.py:15   ``TOTAL ENERGY`` 块 > SUMMARY 的 ``:: total energy`` > opt 周期能量行
cloudra/tools/core/xtb_frames.py:16   （截断日志时最后的周期能量是最佳估计）。
cloudra/tools/core/xtb_frames.py:17 """
cloudra/tools/core/xtb_frames.py:18
cloudra/tools/core/xtb_frames.py:19 from __future__ import annotations
cloudra/tools/core/xtb_frames.py:20
cloudra/tools/core/xtb_frames.py:21 import re
cloudra/tools/core/xtb_frames.py:22
cloudra/tools/core/xtb_frames.py:23 _FLOAT = r"([+-]?(?:\d+(?:\.\d*)?|\.\d+)(?:[DdEe][+-]?\d+)?)"
cloudra/tools/core/xtb_frames.py:24
cloudra/tools/core/xtb_frames.py:25 # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:26 # 终止判定与优化收敛（文本权威）
cloudra/tools/core/xtb_frames.py:27 # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:28
cloudra/tools/core/xtb_frames.py:29 # xtb 正常终止标记：作业成败的文本权威判据（不依赖 cclib）。
cloudra/tools/core/xtb_frames.py:30 NORMAL_TERMINATION_MARKER = "normal termination of xtb"
cloudra/tools/core/xtb_frames.py:31
cloudra/tools/core/xtb_frames.py:32
cloudra/tools/core/xtb_frames.py:33 def normal_termination(text: str) -> bool:
cloudra/tools/core/xtb_frames.py:34     """xtb 日志文本是否含正常终止标记（引擎自报）。"""
cloudra/tools/core/xtb_frames.py:35     return NORMAL_TERMINATION_MARKER in text
cloudra/tools/core/xtb_frames.py:36
cloudra/tools/core/xtb_frames.py:37
cloudra/tools/core/xtb_frames.py:38 # 优化收敛标记：未收敛（跑满最大步数）时 xtb 仍打印 normal termination，
cloudra/tools/core/xtb_frames.py:39 # 终止标记不构成收敛判据（GH #21 同款语义）。两个标记互斥：FAILED 行是
cloudra/tools/core/xtb_frames.py:40 # "CONVERGE GEOMETRY OPTIMIZATION"，
cloudra/tools/core/xtb_frames.py:41 # 不含 "GEOMETRY OPTIMIZATION CONVERGED" 子串。
cloudra/tools/core/xtb_frames.py:42 _OPT_CONVERGED_MARKER = "GEOMETRY OPTIMIZATION CONVERGED"
cloudra/tools/core/xtb_frames.py:43 _OPT_NOT_CONVERGED_MARKER = "FAILED TO CONVERGE GEOMETRY OPTIMIZATION"
cloudra/tools/core/xtb_frames.py:44
cloudra/tools/core/xtb_frames.py:45
cloudra/tools/core/xtb_frames.py:46 def opt_converged(text: str) -> bool | None:
```

```
    """几何优化的收敛状态（文本权威）。
    True=收敛，False=达到最大步数未收敛，None=非优化任务或日志判不出
    （两个标记都不出现，如中途被 kill 的优化日志）。
    """
    if _OPT_NOT_CONVERGED_MARKER in text:
        return False
    if _OPT_CONVERGED_MARKER in text:
        return True
    return None


# ---------------------------------------------------------------------------
# 能量 / HOMO-LUMO gap / 偶极
# ---------------------------------------------------------------------------

# 最终块 "| TOTAL ENERGY  -5.0703... Eh"（大小写敏感，避免误中周期行）
_FINAL_ENERGY_RE = re.compile(rf"\| TOTAL ENERGY\s+{_FLOAT}\s+Eh")
# SUMMARY 块 ":: total energy  -5.0703... Eh"
_SUMMARY_ENERGY_RE = re.compile(rf"::\s*total energy\s+{_FLOAT}\s+Eh")
# opt 周期能量行 " * total energy  :    -5.0703... Eh"
_CYCLE_ENERGY_RE = re.compile(rf"\*\s*total energy\s*:\s+{_FLOAT}\s+Eh")


def final_energy_hartree(text: str) -> float | None:
    """取最终能量（Eh），按文本位置取最后出现的能量行。

    优先级由位置自然给出：最终 TOTAL ENERGY 块在文件尾部；截断/未收敛的
    日志没有最终块时，最后的 SUMMARY 能量或 opt 周期能量行是最佳估计
    （GH #22 同款语义）。
    """
    best_pos = -1
    energy: float | None = None
    for pattern in (_FINAL_ENERGY_RE, _SUMMARY_ENERGY_RE, _CYCLE_ENERGY_RE):
        for match in pattern.finditer(text):
            if match.start() > best_pos:
                best_pos = match.start()
                energy = float(match.group(1).replace("D", "E").replace("d", "e"))
    return energy


_HOMO_LUMO_GAP_RE = re.compile(rf"HOMO-LUMO GAP\s+{_FLOAT}\s+eV", re.IGNORECASE)


def homo_lumo_gap_ev(text: str) -> float | None:
    """HOMO-LUMO gap（eV；最终块/SUMMARY 同值，取第一个命中）。"""
    m = _HOMO_LUMO_GAP_RE.search(text)
    return float(m.group(1)) if m else None


# 偶极矩 "full:  x  y  z  tot"（Property Printout 的分子偶极行）
_DIPOLE_FULL_RE = re.compile(rf"full:\s+{_FLOAT}\s+{_FLOAT}\s+{_FLOAT}\s+{_FLOAT}")
```

```
cloudra/tools/core/xtb_frames.py:99
cloudra/tools/core/xtb_frames.py:100
cloudra/tools/core/xtb_frames.py:101    def dipole_debye(text: str) -> float | None:
cloudra/tools/core/xtb_frames.py:102        """分子偶极矩模（Debye；"full:" 行第四列）。"""
cloudra/tools/core/xtb_frames.py:103        m = _DIPOLE_FULL_RE.search(text)
cloudra/tools/core/xtb_frames.py:104        return float(m.group(4)) if m else None
cloudra/tools/core/xtb_frames.py:105
cloudra/tools/core/xtb_frames.py:106
cloudra/tools/core/xtb_frames.py:107    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:108    # 热化学（THERMODYNAMIC 块 + 最终块的 ENTHALPY/FREE ENERGY 行）
cloudra/tools/core/xtb_frames.py:109    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:110
cloudra/tools/core/xtb_frames.py:111    # THERMODYNAMIC 块："::  total free energy  −25.4987... Eh" 等
cloudra/tools/core/xtb_frames.py:112    _THERMO_FREE_ENERGY_RE = re.compile(rf"::\s*total free energy\s+{_FLOAT}\s+Eh")
cloudra/tools/core/xtb_frames.py:113    _THERMO_ZPE_RE = re.compile(rf"::\s*zero point energy\s+{_FLOAT}\s+Eh")
cloudra/tools/core/xtb_frames.py:114    _THERMO_G_RRHO_RE = re.compile(rf"::\s*G\(RRHO\) contrib\.\s+{_FLOAT}\s+Eh")
cloudra/tools/core/xtb_frames.py:115    # 最终块 "| TOTAL ENTHALPY  −25.4499... Eh"（hess 日志才有；TOTAL ENERGY /
cloudra/tools/core/xtb_frames.py:116    # TOTAL FREE ENERGY 已由能量/free-energy 正则覆盖，注意前缀不互相误中）
cloudra/tools/core/xtb_frames.py:117    _FINAL_ENTHALPY_RE = re.compile(rf"\| TOTAL ENTHALPY\s+{_FLOAT}\s+Eh")
cloudra/tools/core/xtb_frames.py:118
cloudra/tools/core/xtb_frames.py:119
cloudra/tools/core/xtb_frames.py:120    def _last_float(pattern: re.Pattern, text: str) -> float | None:
cloudra/tools/core/xtb_frames.py:121        match = None
cloudra/tools/core/xtb_frames.py:122        for match in pattern.finditer(text):
cloudra/tools/core/xtb_frames.py:123            pass
cloudra/tools/core/xtb_frames.py:124        if match is None:
cloudra/tools/core/xtb_frames.py:125            return None
cloudra/tools/core/xtb_frames.py:126        return float(match.group(1).replace("D", "E").replace("d", "e"))
cloudra/tools/core/xtb_frames.py:127
cloudra/tools/core/xtb_frames.py:128
cloudra/tools/core/xtb_frames.py:129    def thermochemistry(text: str) -> dict[str, float]:
cloudra/tools/core/xtb_frames.py:130        """提取 xtb hess 日志的热化学量（Eh），键名与 log_parser 输出字段对齐。
cloudra/tools/core/xtb_frames.py:131
cloudra/tools/core/xtb_frames.py:132        cclib 对 xtb --hess 日志的热化学属性给出荒谬值（实锤：freeenergy 报
cloudra/tools/core/xtb_frames.py:133        −693 Eh），权威值在 THERMODYNAMIC 块与最终块：
cloudra/tools/core/xtb_frames.py:134
cloudra/tools/core/xtb_frames.py:135        - ``total_gibbs_free_energy_hartree`` ← ``::  total free energy``；
cloudra/tools/core/xtb_frames.py:136        - ``total_enthalpy_hartree`` ← 最终块 ``TOTAL ENTHALPY``；
cloudra/tools/core/xtb_frames.py:137        - ``zero_point_correction_hartree`` ← ``::  zero point energy``；
cloudra/tools/core/xtb_frames.py:138        - ``thermal_gibbs_correction_hartree`` ← ``::  G(RRHO) contrib.``。
cloudra/tools/core/xtb_frames.py:139
cloudra/tools/core/xtb_frames.py:140        无热化学块（非 freq 任务）时返回空 dict——缺数据保持缺失，不编数。
cloudra/tools/core/xtb_frames.py:141        """
cloudra/tools/core/xtb_frames.py:142        thermo: dict[str, float] = {}
cloudra/tools/core/xtb_frames.py:143        free_energy = _last_float(_THERMO_FREE_ENERGY_RE, text)
cloudra/tools/core/xtb_frames.py:144        if free_energy is not None:
cloudra/tools/core/xtb_frames.py:145            thermo["total_gibbs_free_energy_hartree"] = free_energy
cloudra/tools/core/xtb_frames.py:146        enthalpy = _last_float(_FINAL_ENTHALPY_RE, text)
cloudra/tools/core/xtb_frames.py:147        if enthalpy is not None:
cloudra/tools/core/xtb_frames.py:148            thermo["total_enthalpy_hartree"] = enthalpy
cloudra/tools/core/xtb_frames.py:149        zpe = _last_float(_THERMO_ZPE_RE, text)
cloudra/tools/core/xtb_frames.py:150        if zpe is not None:
```

```
cloudra/tools/core/xtb_frames.py:151            thermo["zero_point_correction_hartree"] = zpe
cloudra/tools/core/xtb_frames.py:152        g_rrho = _last_float(_THERMO_G_RRHO_RE, text)
cloudra/tools/core/xtb_frames.py:153        if g_rrho is not None:
cloudra/tools/core/xtb_frames.py:154            thermo["thermal_gibbs_correction_hartree"] = g_rrho
cloudra/tools/core/xtb_frames.py:155        return thermo
cloudra/tools/core/xtb_frames.py:156
cloudra/tools/core/xtb_frames.py:157
cloudra/tools/core/xtb_frames.py:158    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:159    # 频率（projected vibrational frequencies 的 eigval 行）
cloudra/tools/core/xtb_frames.py:160    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:161
cloudra/tools/core/xtb_frames.py:162    _EIGVAL_LINE_RE = re.compile(r"^\s*eigval\s*:\s*(.*)$")
cloudra/tools/core/xtb_frames.py:163    # 虚频的另一种写法：数值带 i 后缀（如 "68.66i"），语义等同负值
cloudra/tools/core/xtb_frames.py:164    _EIGVAL_TOKEN_RE = re.compile(rf"^{_FLOAT}(i?)$", re.IGNORECASE)
cloudra/tools/core/xtb_frames.py:165
cloudra/tools/core/xtb_frames.py:166
cloudra/tools/core/xtb_frames.py:167    def _eigval_tokens(line_body: str) -> list[float]:
cloudra/tools/core/xtb_frames.py:168        values: list[float] = []
cloudra/tools/core/xtb_frames.py:169        for tok in line_body.split():
cloudra/tools/core/xtb_frames.py:170            m = _EIGVAL_TOKEN_RE.match(tok)
cloudra/tools/core/xtb_frames.py:171            if m is None:
cloudra/tools/core/xtb_frames.py:172                continue
cloudra/tools/core/xtb_frames.py:173            value = float(m.group(1).replace("D", "E").replace("d", "e"))
cloudra/tools/core/xtb_frames.py:174            values.append(-abs(value) if m.group(2) else value)
cloudra/tools/core/xtb_frames.py:175        return values
cloudra/tools/core/xtb_frames.py:176
cloudra/tools/core/xtb_frames.py:177
cloudra/tools/core/xtb_frames.py:178    def vibfreqs_from_text(text: str) -> list[float]:
cloudra/tools/core/xtb_frames.py:179        """解析最后一个 projected vibrational frequencies 块的频率（cm⁻¹）。
cloudra/tools/core/xtb_frames.py:180
cloudra/tools/core/xtb_frames.py:181        负值=虚频（i 后缀写法同样解析为负值）；保留打头的平动/转动零频模式
cloudra/tools/core/xtb_frames.py:182        （与 orca_frames 同款语义），调用方按符号拆分实频/虚频即可。无频率块
cloudra/tools/core/xtb_frames.py:183        时返回空列表——调用方须保持 None 语义，不得当成零虚频。
cloudra/tools/core/xtb_frames.py:184        """
cloudra/tools/core/xtb_frames.py:185        idx = text.rfind("projected vibrational frequencies")
cloudra/tools/core/xtb_frames.py:186        if idx < 0:
cloudra/tools/core/xtb_frames.py:187            return []
cloudra/tools/core/xtb_frames.py:188        freqs: list[float] = []
cloudra/tools/core/xtb_frames.py:189        for line in text[idx:].splitlines()[1:]:
cloudra/tools/core/xtb_frames.py:190            m = _EIGVAL_LINE_RE.match(line)
cloudra/tools/core/xtb_frames.py:191            if m is None:
cloudra/tools/core/xtb_frames.py:192                if freqs:
cloudra/tools/core/xtb_frames.py:193                    break  # eigval 行连续成块，块外即停
cloudra/tools/core/xtb_frames.py:194                continue
cloudra/tools/core/xtb_frames.py:195            freqs.extend(_eigval_tokens(m.group(1)))
cloudra/tools/core/xtb_frames.py:196        return freqs
cloudra/tools/core/xtb_frames.py:197
cloudra/tools/core/xtb_frames.py:198
cloudra/tools/core/xtb_frames.py:199    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:200    # 任务类型识别
cloudra/tools/core/xtb_frames.py:201    # ---------------------------------------------------------------------------
cloudra/tools/core/xtb_frames.py:202
```

```
cloudra/tools/core/xtb_frames.py:203    # opt 周期分隔行 "............................. CYCLE    8 ..........."
cloudra/tools/core/xtb_frames.py:204    _OPT_CYCLE_RE = re.compile(r"^\.{3,}\s*CYCLE\s+\d+\s*\.{3,}\s*$", re.M)
cloudra/tools/core/xtb_frames.py:205
cloudra/tools/core/xtb_frames.py:206
cloudra/tools/core/xtb_frames.py:207    def detect_job_types_from_text(text: str) -> set[str]:
cloudra/tools/core/xtb_frames.py:208        """识别 xtb 日志的任务类型，返回 {"opt","freq","sp"} 的子集。
cloudra/tools/core/xtb_frames.py:209
cloudra/tools/core/xtb_frames.py:210        xtb stdout 不回显 CLI 标志，只能从输出节判定：CYCLE 迭代行/收敛标记
cloudra/tools/core/xtb_frames.py:211        → opt；projected vibrational frequencies 块 → freq；非优化即 sp。
cloudra/tools/core/xtb_frames.py:212        TS 搜索无独立输出节，判不出（xtb 的 saddle 优化走外部约束文件）。
cloudra/tools/core/xtb_frames.py:213        """
cloudra/tools/core/xtb_frames.py:214        jobs: set[str] = set()
cloudra/tools/core/xtb_frames.py:215        if (
cloudra/tools/core/xtb_frames.py:216            _OPT_CONVERGED_MARKER in text
cloudra/tools/core/xtb_frames.py:217            or _OPT_NOT_CONVERGED_MARKER in text
cloudra/tools/core/xtb_frames.py:218            or _OPT_CYCLE_RE.search(text)
cloudra/tools/core/xtb_frames.py:219        ):
cloudra/tools/core/xtb_frames.py:220            jobs.add("opt")
cloudra/tools/core/xtb_frames.py:221        if "projected vibrational frequencies" in text:
cloudra/tools/core/xtb_frames.py:222            jobs.add("freq")
cloudra/tools/core/xtb_frames.py:223        if "opt" not in jobs:
cloudra/tools/core/xtb_frames.py:224            jobs.add("sp")
cloudra/tools/core/xtb_frames.py:225        return jobs
cloudra/tools/core/xtb_path_runner.py:1 """xTB MTD 路径搜索运行器：TS 初猜的便宜端（SOP `guess` 阶段）。
cloudra/tools/core/xtb_path_runner.py:2
cloudra/tools/core/xtb_path_runner.py:3 **事实核对（本机 xtb 6.7.1 实测，不是推断）**
cloudra/tools/core/xtb_path_runner.py:4
cloudra/tools/core/xtb_path_runner.py:5 - xTB **没有 NEB**。它有 ``--path [FILE]``：元动力学（MTD）路径搜索，从输入几何走到
cloudra/tools/core/xtb_path_runner.py:6   给定的产物几何。所以本模块叫 path 不叫 neb。
cloudra/tools/core/xtb_path_runner.py:7 - 调用形式：``xtb reactant.xyz --path product.xyz``。**两个几何必需**，元素序列必须一致。
cloudra/tools/core/xtb_path_runner.py:8 - 产出（持久运行目录里原地保留）：
cloudra/tools/core/xtb_path_runner.py:9
cloudra/tools/core/xtb_path_runner.py:10      ====================  ================================================
cloudra/tools/core/xtb_path_runner.py:11      文件                  内容
cloudra/tools/core/xtb_path_runner.py:12      ====================  ================================================
cloudra/tools/core/xtb_path_runner.py:13      ``xtbpath.xyz``       路径的逐帧几何（含起点）。注释行 ``energy: X`` 是
cloudra/tools/core/xtb_path_runner.py:14                            **相对输入几何的能量，单位 kcal/mol**
cloudra/tools/core/xtb_path_runner.py:15      ``xtbpath_ts.xyz``    MTD 找到的 TS **候选**。注释行 ``energy: X`` 是
cloudra/tools/core/xtb_path_runner.py:16                            **绝对能量，单位 Eh**
cloudra/tools/core/xtb_path_runner.py:17      ``xtbopt.log``        产物侧预优化的轨迹
cloudra/tools/core/xtb_path_runner.py:18      ``xtbopt.xyz`` 等     引擎其余中间产物
cloudra/tools/core/xtb_path_runner.py:19      ====================  ================================================
cloudra/tools/core/xtb_path_runner.py:20
cloudra/tools/core/xtb_path_runner.py:21      **两个文件的注释行单位不同，这是最容易静默出错的地方**：实测
cloudra/tools/core/xtb_path_runner.py:22      ``xtbpath_ts.xyz`` 的 ``-4.987083865354`` 与单点 ``TOTAL ENERGY
    -4.987083865355 Eh``
cloudra/tools/core/xtb_path_runner.py:23      一致到 1e-12；而 ``xtbpath.xyz`` 里某帧注释 ``52.271`` 对应的真实能量差是
cloudra/tools/core/xtb_path_runner.py:24      ``-4.987083865354 - (-5.070383095108) = 0.083299 Eh = 52.272 kcal/mol``。
cloudra/tools/core/xtb_path_runner.py:25      因此本模块把两者**分别标注**（``rel_energy_kcal`` / ``ts_candidate_energy_eh``），
cloudra/tools/core/xtb_path_runner.py:26      绝不放进同一个字段。
cloudra/tools/core/xtb_path_runner.py:27
cloudra/tools/core/xtb_path_runner.py:28    **这个结果只是初猜**：``xtbpath_ts.xyz`` 是 MTD 采到的最高点附近结构，不是鞍点，
cloudra/tools/core/xtb_path_runner.py:29    更不是"已验证"。验证仍归频率/IRC 判定。
```

```
cloudra/tools/core/xtb_path_runner.py:30
cloudra/tools/core/xtb_path_runner.py:31    **随机性**：MTD 是随机的，同一次输入重跑路径不同。因此结果按**采样**看待，命令与
cloudra/tools/core/xtb_path_runner.py:32    stdout 全部落盘以备复查。
cloudra/tools/core/xtb_path_runner.py:33
cloudra/tools/core/xtb_path_runner.py:34    **路径精修必须跑到引擎自己判收敛**（本机 xtb 6.7.1 实测，逐字证据与可重跑脚本在
cloudra/tools/core/xtb_path_runner.py:35    ``docs/evidence/ts_guess_mtd_artifact/``）：xTB 的路径是**迭代精修**的，而它的内建上限
只有
cloudra/tools/core/xtb_path_runner.py:36    3 轮，3 轮后会打印 ``terminated because max. # cycles reached`` —— 那不是收敛，是**
被截断**。
cloudra/tools/core/xtb_path_runner.py:37    截断路径的最高帧落在反应物自己的盆地里（实测：最高帧离起点只有 21°，相对能 44.865 kcal/mol
，
cloudra/tools/core/xtb_path_runner.py:38    拿去做 ORCA ``OptTS`` 跑 56 轮不收敛、一路滑回极小值）。把上限放开后，同一个端点对在第 31
轮
cloudra/tools/core/xtb_path_runner.py:39    由引擎自己判收敛（``terminated because grad at TS <threshold``），barrier 降到
18.299 kcal/mol，
cloudra/tools/core/xtb_path_runner.py:40    最高帧落在扭转极大上，``OptTS`` 9 轮即收敛到一阶鞍点。因此本模块：
cloudra/tools/core/xtb_path_runner.py:41
cloudra/tools/core/xtb_path_runner.py:42    - **总是**写一份 ``$path`` 输入并把 ``nrun`` 上限显式传给引擎（内建默认 3 轮不能用）；
cloudra/tools/core/xtb_path_runner.py:43    - 把这两句终止语**分开分类**：``converged`` / ``cycles_exhausted``；
cloudra/tools/core/xtb_path_runner.py:44    - 触到上限一律报 ``path_not_converged`` 且 ``usable_as_guess=False``，绝不把截断路径
当可用初猜。
cloudra/tools/core/xtb_path_runner.py:45    """
cloudra/tools/core/xtb_path_runner.py:46
cloudra/tools/core/xtb_path_runner.py:47    from __future__ import annotations
cloudra/tools/core/xtb_path_runner.py:48
cloudra/tools/core/xtb_path_runner.py:49    import os
cloudra/tools/core/xtb_path_runner.py:50    import signal
cloudra/tools/core/xtb_path_runner.py:51    import subprocess
cloudra/tools/core/xtb_path_runner.py:52    from typing import Literal, Optional
cloudra/tools/core/xtb_path_runner.py:53
cloudra/tools/core/xtb_path_runner.py:54    from pydantic import BaseModel, Field
cloudra/tools/core/xtb_path_runner.py:55
cloudra/tools/core/xtb_path_runner.py:56    from cloudra.persistence.fs_utils import atomic_write_text
cloudra/tools/core/xtb_path_runner.py:57
cloudra/tools/core/xtb_path_runner.py:58    PATH_XYZ_NAME = "xtbpath.xyz"
cloudra/tools/core/xtb_path_runner.py:59    TS_XYZ_NAME = "xtbpath_ts.xyz"
cloudra/tools/core/xtb_path_runner.py:60    RUNS_DIR_NAME = "xtb_path_runs"
cloudra/tools/core/xtb_path_runner.py:61    MAX_EVIDENCE_LINES = 20
cloudra/tools/core/xtb_path_runner.py:62
cloudra/tools/core/xtb_path_runner.py:63    # --- xtb 6.7.1 的逐字标记（精确比对，不做模糊匹配；本模块不含正则） ---
cloudra/tools/core/xtb_path_runner.py:64    MARKER_NORMAL_TERMINATION = "normal termination of xtb"
cloudra/tools/core/xtb_path_runner.py:65    MARKER_FINISHED_RUN = " * finished run on"
cloudra/tools/core/xtb_path_runner.py:66    MARKER_PATH_FINDER = " path finder:"
cloudra/tools/core/xtb_path_runner.py:67    #: 路径精修的两句终止语。前者是引擎判的收敛，后者是**触到轮数上限被截断**，二者绝不等价。
cloudra/tools/core/xtb_path_runner.py:68    MARKER_PATH_CONVERGED = "terminated because grad at TS <threshold"
cloudra/tools/core/xtb_path_runner.py:69    MARKER_PATH_CYCLES_EXHAUSTED = "terminated because max. # cycles reached"
cloudra/tools/core/xtb_path_runner.py:70    #: 每轮精修打印的 barrier（第一列是空格对齐，逐字保留）
cloudra/tools/core/xtb_path_runner.py:71    MARKER_FORWARD_BARRIER = "forward  barrier (kcal)"
cloudra/tools/core/xtb_path_runner.py:72    MARKER_BACKWARD_BARRIER = "backward barrier (kcal)"
cloudra/tools/core/xtb_path_runner.py:73
cloudra/tools/core/xtb_path_runner.py:74    #: 路径精修的轮数上限（``$path nrun``）。这**不是**收敛判据，只是机时上限：收敛由引擎自己判
cloudra/tools/core/xtb_path_runner.py:75    #: （见模块 docstring 的实测）。默认给足，让引擎跑到自己的收敛判据为止。
cloudra/tools/core/xtb_path_runner.py:76    DEFAULT_REFINEMENT_RUNS = 100
cloudra/tools/core/xtb_path_runner.py:77    #: ``$path`` 输入文件名（与命令行一起落盘备查）
cloudra/tools/core/xtb_path_runner.py:78    PATH_INPUT_NAME = "path.inp"
cloudra/tools/core/xtb_path_runner.py:79
cloudra/tools/core/xtb_path_runner.py:80    #: 初猜来源标识（SOP §2：初猜是怎么来的必须写明，不许静默降级）。
cloudra/tools/core/xtb_path_runner.py:81    PATH_KIND_XTB = "xtb_path"
```

```
cloudra/tools/core/xtb_path_runner.py:82    PATH_KIND_IDPP = "idpp_fallback"
cloudra/tools/core/xtb_path_runner.py:83
cloudra/tools/core/xtb_path_runner.py:84    PathTermination = Literal[
cloudra/tools/core/xtb_path_runner.py:85        "completed",
cloudra/tools/core/xtb_path_runner.py:86        "executable_missing",
cloudra/tools/core/xtb_path_runner.py:87        "timeout",
cloudra/tools/core/xtb_path_runner.py:88        "invalid_endpoints",
cloudra/tools/core/xtb_path_runner.py:89        "incomplete_artifacts",
cloudra/tools/core/xtb_path_runner.py:90        "path_not_converged",
cloudra/tools/core/xtb_path_runner.py:91        "failed",
cloudra/tools/core/xtb_path_runner.py:92        "unknown_failure",
cloudra/tools/core/xtb_path_runner.py:93    ]
cloudra/tools/core/xtb_path_runner.py:94
cloudra/tools/core/xtb_path_runner.py:95    #: 引擎自己报的路径精修结局。``""`` = 日志里没有这两句终止语（本模块按未知处理，不默认成功）。
cloudra/tools/core/xtb_path_runner.py:96    RefinementTermination = Literal["", "converged", "cycles_exhausted"]
cloudra/tools/core/xtb_path_runner.py:97
cloudra/tools/core/xtb_path_runner.py:98
cloudra/tools/core/xtb_path_runner.py:99    class XtbPathRequest(BaseModel):
cloudra/tools/core/xtb_path_runner.py:100       """一次 MTD 路径搜索的请求。"""
cloudra/tools/core/xtb_path_runner.py:101
cloudra/tools/core/xtb_path_runner.py:102       reactant_xyz: str = Field(description="反应物几何（单帧 xyz 文本）")
cloudra/tools/core/xtb_path_runner.py:103       product_xyz: str = Field(description="产物几何（单帧 xyz 文本）；元素序列必须与反应物一致")
cloudra/tools/core/xtb_path_runner.py:104       output_dir: str = Field(description="持久运行目录的父目录；每次运行在其 xtb_path_runs/ 下新建")
cloudra/tools/core/xtb_path_runner.py:105       charge: int = Field(default=0, description="总电荷（--chrg）")
cloudra/tools/core/xtb_path_runner.py:106       multiplicity: int = Field(default=1, description="自旋多重度 2S+1，透传为 --uhf = multiplicity-1")
cloudra/tools/core/xtb_path_runner.py:107       timeout: int = Field(default=900, description="子进程耗时上限（秒）；超时只终止本次运行自己的进程组")
cloudra/tools/core/xtb_path_runner.py:108       refinement_runs: int = Field(
cloudra/tools/core/xtb_path_runner.py:109           default=DEFAULT_REFINEMENT_RUNS, ge=1,
cloudra/tools/core/xtb_path_runner.py:110           description="路径精修的轮数上限（$path nrun）。**只是机时上限**：收敛由引擎自己判，"
cloudra/tools/core/xtb_path_runner.py:111                       "触到这个上限即报 path_not_converged。实测内建默认 3 轮远不够收敛，"
cloudra/tools/core/xtb_path_runner.py:112                       "不要把它调小",
cloudra/tools/core/xtb_path_runner.py:113       )
cloudra/tools/core/xtb_path_runner.py:114       run_label: str = Field(default="xtbpath", description="运行目录前缀")
cloudra/tools/core/xtb_path_runner.py:115       extra_args: list[str] = Field(default_factory=list, description="追加到命令行的原生选项")
cloudra/tools/core/xtb_path_runner.py:116
cloudra/tools/core/xtb_path_runner.py:117
cloudra/tools/core/xtb_path_runner.py:118   class XtbArtifact(BaseModel):
cloudra/tools/core/xtb_path_runner.py:119       name: str
cloudra/tools/core/xtb_path_runner.py:120       path: str
cloudra/tools/core/xtb_path_runner.py:121       size_bytes: int
cloudra/tools/core/xtb_path_runner.py:122
cloudra/tools/core/xtb_path_runner.py:123
cloudra/tools/core/xtb_path_runner.py:124   class PathFrame(BaseModel):
cloudra/tools/core/xtb_path_runner.py:125       """路径上的一帧。"""
cloudra/tools/core/xtb_path_runner.py:126
cloudra/tools/core/xtb_path_runner.py:127       frame_index: int
cloudra/tools/core/xtb_path_runner.py:128       rel_energy_kcal: Optional[float] = Field(
cloudra/tools/core/xtb_path_runner.py:129           default=None,
cloudra/tools/core/xtb_path_runner.py:130           description="文件注释里的能量，单位**已核实为 kcal/mol**，相对输入几何；取不到时为 None",
cloudra/tools/core/xtb_path_runner.py:131       )
cloudra/tools/core/xtb_path_runner.py:132       positions: list[list[float]]
cloudra/tools/core/xtb_path_runner.py:133
```

```
cloudra/tools/core/xtb_path_runner.py:134
cloudra/tools/core/xtb_path_runner.py:135  class XtbPathResult(BaseModel):
cloudra/tools/core/xtb_path_runner.py:136      """路径搜索结果。进程退出与"能不能用来当初猜"分开表达。"""
cloudra/tools/core/xtb_path_runner.py:137
cloudra/tools/core/xtb_path_runner.py:138      run_dir: str
cloudra/tools/core/xtb_path_runner.py:139      command: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:140      returncode: Optional[int] = None
cloudra/tools/core/xtb_path_runner.py:141      path_kind: Literal["xtb_path"] = PATH_KIND_XTB
cloudra/tools/core/xtb_path_runner.py:142      termination_reason: PathTermination = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:143      refinement_runs: int = Field(default=0, description="本次传给引擎的路径精修轮数
上限")
cloudra/tools/core/xtb_path_runner.py:144      refinement_termination: RefinementTermination = Field(
cloudra/tools/core/xtb_path_runner.py:145          default="",
cloudra/tools/core/xtb_path_runner.py:146          description="引擎自己报的精修结局：converged（引擎判收敛）/ cycles_exhausted
（触顶截断）",
cloudra/tools/core/xtb_path_runner.py:147      )
cloudra/tools/core/xtb_path_runner.py:148      usable_as_guess: bool = Field(
cloudra/tools/core/xtb_path_runner.py:149          default=False,
cloudra/tools/core/xtb_path_runner.py:150          description="是否产出了可用的初猜（路径 + TS 候选齐全且几何自洽）。"
cloudra/tools/core/xtb_path_runner.py:151                      "**这不等于已验证**：初猜仍需频率/IRC 判定",
cloudra/tools/core/xtb_path_runner.py:152      )
cloudra/tools/core/xtb_path_runner.py:153      symbols: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:154      frames: list[PathFrame] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:155      ts_candidate_energy_eh: Optional[float] = Field(
cloudra/tools/core/xtb_path_runner.py:156          default=None, description="TS 候选的**绝对**能量（Eh）；与路径帧的相对 kcal/
mol 不是一个字段"
cloudra/tools/core/xtb_path_runner.py:157      )
cloudra/tools/core/xtb_path_runner.py:158      ts_candidate_positions: list[list[float]] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:159      reactant_path: str = ""
cloudra/tools/core/xtb_path_runner.py:160      product_path: str = ""
cloudra/tools/core/xtb_path_runner.py:161      command_path: str = ""
cloudra/tools/core/xtb_path_runner.py:162      path_input_path: str = ""
cloudra/tools/core/xtb_path_runner.py:163      versions_path: str = ""
cloudra/tools/core/xtb_path_runner.py:164      stdout_path: str = ""
cloudra/tools/core/xtb_path_runner.py:165      stderr_path: str = ""
cloudra/tools/core/xtb_path_runner.py:166      path_xyz_path: str = ""
cloudra/tools/core/xtb_path_runner.py:167      ts_xyz_path: str = ""
cloudra/tools/core/xtb_path_runner.py:168      artifacts: list[XtbArtifact] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:169      evidence: list[str] = Field(default_factory=list, description="命中的逐字标记
行（原样保留）")
cloudra/tools/core/xtb_path_runner.py:170      diagnostics: list[str] = Field(default_factory=list)
cloudra/tools/core/xtb_path_runner.py:171      message: str = ""
cloudra/tools/core/xtb_path_runner.py:172
cloudra/tools/core/xtb_path_runner.py:173
cloudra/tools/core/xtb_path_runner.py:174  def build_path_input(refinement_runs: int) -> str:
cloudra/tools/core/xtb_path_runner.py:175      """``$path`` 输入块：把精修轮数上限显式交给引擎，**不使用内建默认 3 轮**。
cloudra/tools/core/xtb_path_runner.py:176
cloudra/tools/core/xtb_path_runner.py:177      本机 6.7.1 实测：``$path`` 块被读取；它认得的键是 ``nrun`` / ``npoint`` / ``kpush
`` /
cloudra/tools/core/xtb_path_runner.py:178      ``kpull``，其余（``npoints`` / ``alpha`` / ``step`` / ``time`` / ``seed``
等）会被逐字拒掉
cloudra/tools/core/xtb_path_runner.py:179      （``the key '...' is not recognized by path``），因此这里只写实测可用的键。
cloudra/tools/core/xtb_path_runner.py:180      """
cloudra/tools/core/xtb_path_runner.py:181      if refinement_runs < 1:
cloudra/tools/core/xtb_path_runner.py:182          raise ValueError(f"refinement_runs must be >= 1, got {refinement_runs
}")
cloudra/tools/core/xtb_path_runner.py:183      return f"$path\n   nrun={refinement_runs}\n$end\n"
cloudra/tools/core/xtb_path_runner.py:184
cloudra/tools/core/xtb_path_runner.py:185
```

```
cloudra/tools/core/xtb_path_runner.py:186 def build_xtb_path_command(request: XtbPathRequest) -> list[str]:
cloudra/tools/core/xtb_path_runner.py:187     """构造命令行。电荷与自旋显式透传，绝不依赖引擎默认推断。"""
cloudra/tools/core/xtb_path_runner.py:188     if request.multiplicity < 1:
cloudra/tools/core/xtb_path_runner.py:189         raise ValueError(f"multiplicity must be >= 1 (2S+1), got {request.multiplicity}")
cloudra/tools/core/xtb_path_runner.py:190     return [
cloudra/tools/core/xtb_path_runner.py:191         "xtb", "reactant.xyz", "--path", "product.xyz", "--input", PATH_INPUT_NAME,
cloudra/tools/core/xtb_path_runner.py:192         "--chrg", str(request.charge), "--uhf", str(request.multiplicity - 1),
cloudra/tools/core/xtb_path_runner.py:193         *request.extra_args,
cloudra/tools/core/xtb_path_runner.py:194     ]
cloudra/tools/core/xtb_path_runner.py:195
cloudra/tools/core/xtb_path_runner.py:196
cloudra/tools/core/xtb_path_runner.py:197 def _next_run_dir(output_dir: str, run_label: str) -> str:
cloudra/tools/core/xtb_path_runner.py:198     safe_label = "".join(ch if (ch.isalnum() or ch in "-_") else "_" for ch in run_label) or "run"
cloudra/tools/core/xtb_path_runner.py:199     runs_root = os.path.join(output_dir, RUNS_DIR_NAME)
cloudra/tools/core/xtb_path_runner.py:200     os.makedirs(runs_root, exist_ok=True)
cloudra/tools/core/xtb_path_runner.py:201     used = set()
cloudra/tools/core/xtb_path_runner.py:202     for name in os.listdir(runs_root):
cloudra/tools/core/xtb_path_runner.py:203         head, _, tail = name.rpartition("_")
cloudra/tools/core/xtb_path_runner.py:204         if head == safe_label and tail.isdigit():
cloudra/tools/core/xtb_path_runner.py:205             used.add(int(tail))
cloudra/tools/core/xtb_path_runner.py:206     index = 1
cloudra/tools/core/xtb_path_runner.py:207     while index in used:
cloudra/tools/core/xtb_path_runner.py:208         index += 1
cloudra/tools/core/xtb_path_runner.py:209     run_dir = os.path.join(runs_root, f"{safe_label}_{index:04d}")
cloudra/tools/core/xtb_path_runner.py:210     os.makedirs(run_dir, exist_ok=False)
cloudra/tools/core/xtb_path_runner.py:211     return run_dir
cloudra/tools/core/xtb_path_runner.py:212
cloudra/tools/core/xtb_path_runner.py:213
cloudra/tools/core/xtb_path_runner.py:214 def _capture_versions() -> str:
cloudra/tools/core/xtb_path_runner.py:215     try:
cloudra/tools/core/xtb_path_runner.py:216         proc = subprocess.run(["xtb", "--version"], capture_output=True, text=True, timeout=60)
cloudra/tools/core/xtb_path_runner.py:217     except Exception as exc:  # noqa: BLE001 - 版本取不到不阻塞运行，如实记录
cloudra/tools/core/xtb_path_runner.py:218         return f"xtb --version failed: {type(exc).__name__}: {exc}\n"
cloudra/tools/core/xtb_path_runner.py:219     text = (proc.stdout or "") + (proc.stderr or "")
cloudra/tools/core/xtb_path_runner.py:220     lines = [line.strip() for line in text.splitlines() if "xtb version" in line]
cloudra/tools/core/xtb_path_runner.py:221     return "xtb --version (returncode=%d):\n%s\n" % (
cloudra/tools/core/xtb_path_runner.py:222         proc.returncode, "\n".join(f"    {line}" for line in (lines or ["<no version line found>"])),
cloudra/tools/core/xtb_path_runner.py:223     )
cloudra/tools/core/xtb_path_runner.py:224
cloudra/tools/core/xtb_path_runner.py:225
cloudra/tools/core/xtb_path_runner.py:226 def _symbols_and_positions(xyz_text: str, *, label: str) -> tuple[list[str], list[list[float]]]:
cloudra/tools/core/xtb_path_runner.py:227     """解析单帧 xyz；不完整即 fail-loud。"""
cloudra/tools/core/xtb_path_runner.py:228     lines = xyz_text.splitlines()
cloudra/tools/core/xtb_path_runner.py:229     if not lines:
cloudra/tools/core/xtb_path_runner.py:230         raise ValueError(f"{label}: empty xyz text")
cloudra/tools/core/xtb_path_runner.py:231     try:
cloudra/tools/core/xtb_path_runner.py:232         count = int(lines[0].strip())
cloudra/tools/core/xtb_path_runner.py:233     except ValueError as exc:
cloudra/tools/core/xtb_path_runner.py:234         raise ValueError(f"{label}: first line must be the atom count, got {lines[0]!r}") from exc
cloudra/tools/core/xtb_path_runner.py:235     if count < 1:
cloudra/tools/core/xtb_path_runner.py:236         raise ValueError(f"{label}: declares {count} atoms")
cloudra/tools/core/xtb_path_runner.py:237     if len(lines) < 2 + count:
```

```
            raise ValueError(f"{label}: declares {count} atoms but has too few lines")
        symbols: list[str] = []
        positions: list[list[float]] = []
        for line in lines[2: 2 + count]:
            parts = line.split()
            if len(parts) < 4:
                raise ValueError(f"{label}: malformed coordinate line {line!r}")
            symbols.append(parts[0])
            positions.append([float(parts[1]), float(parts[2]), float(parts[3])])
        return symbols, positions


    def _parse_path_frames(path: str) -> list[PathFrame]:
        """解析 ``xtbpath.xyz``。注释里的能量是**相对 kcal/mol**，取不到就留 None，不猜。"""
        with open(path, encoding="utf-8", errors="replace") as handle:
            lines = handle.read().splitlines()
        frames: list[PathFrame] = []
        index = 0
        while index < len(lines):
            if not lines[index].strip():
                index += 1
                continue
            try:
                count = int(lines[index].strip())
            except ValueError:
                break
            comment = lines[index + 1] if index + 1 < len(lines) else ""
            rel: Optional[float] = None
            tokens = comment.split()
            # 形如 ``energy: <float> xtb: <version>``；逐 token 取，不做模糊匹配
            if len(tokens) >= 2 and tokens[0] == "energy:":
                try:
                    rel = float(tokens[1])
                except ValueError:
                    rel = None
            block = lines[index + 2: index + 2 + count]
            if len(block) != count:
                break
            positions = []
            for line in block:
                parts = line.split()
                if len(parts) < 4:
                    raise ValueError(f"{path}: malformed coordinate line {line!r}")
                positions.append([float(parts[1]), float(parts[2]), float(parts[3])])
            frames.append(PathFrame(frame_index=len(frames), rel_energy_kcal=rel, positions=positions))
            index += 2 + count
        if not frames:
            raise ValueError(f"{path}: no complete frame could be read")
        return frames


    def _parse_ts_candidate(path: str) -> tuple[float, list[list[float]]]:
```

```
    """解析 ``xtbpath_ts.xyz``。注释里的能量是**绝对 Eh**——与路径帧的单位不同。"""
    with open(path, encoding="utf-8", errors="replace") as handle:
        lines = handle.read().splitlines()
    if len(lines) < 3:
        raise ValueError(f"{path}: too few lines for a single frame")
    count = int(lines[0].strip())
    tokens = lines[1].split()
    if len(tokens) < 2 or tokens[0] != "energy:":
        raise ValueError(
            f"{path}: comment line must start with 'energy: <Eh>', got {lines[1]!r}"

        )
    energy = float(tokens[1])
    block = lines[2: 2 + count]
    if len(block) != count:
        raise ValueError(f"{path}: declares {count} atoms but only {len(block)} coordinates follow")
    positions = []
    for line in block:
        parts = line.split()
        if len(parts) < 4:
            raise ValueError(f"{path}: malformed coordinate line {line!r}")
        positions.append([float(parts[1]), float(parts[2]), float(parts[3])])
    return energy, positions


def _list_artifacts(run_dir: str) -> list[XtbArtifact]:
    artifacts: list[XtbArtifact] = []
    for name in sorted(os.listdir(run_dir)):
        path = os.path.join(run_dir, name)
        if os.path.isfile(path):
            artifacts.append(XtbArtifact(name=name, path=path, size_bytes=os.path.getsize(path)))
    return artifacts


class PathMarkers(BaseModel):
    """日志里命中的逐字标记。"""

    evidence: list[str] = Field(default_factory=list)
    normal_termination: bool = False
    path_finder: bool = False
    refinement_termination: RefinementTermination = ""


def _scan_markers(lines: list[str]) -> PathMarkers:
    """精确 token 扫描。两条终止语**分开**判定：收敛与触顶绝不等价。

    barrier 行每轮精修都打印一次，只保留**最后一次**（更靠后的那个才是当前精修程度）。
    """
    markers = PathMarkers()
    last_forward = ""
    last_backward = ""
    for line in lines:
        stripped = line.strip()
```

```
            if MARKER_NORMAL_TERMINATION in stripped:
                markers.evidence.append(stripped)
                markers.normal_termination = True
            elif stripped.startswith(MARKER_FINISHED_RUN.strip()):
                markers.evidence.append(stripped)
            elif MARKER_PATH_FINDER.strip() in stripped:
                markers.evidence.append(stripped)
                markers.path_finder = True
            elif MARKER_PATH_CONVERGED in stripped:
                markers.evidence.append(stripped)
                markers.refinement_termination = "converged"
            elif MARKER_PATH_CYCLES_EXHAUSTED in stripped:
                markers.evidence.append(stripped)
                markers.refinement_termination = "cycles_exhausted"
            elif MARKER_FORWARD_BARRIER in stripped:
                last_forward = stripped
            elif MARKER_BACKWARD_BARRIER in stripped:
                last_backward = stripped
        for extra in (last_forward, last_backward):
            if extra:
                markers.evidence.append(extra)
        markers.evidence = markers.evidence[:MAX_EVIDENCE_LINES]
        return markers


def run_xtb_path(request: XtbPathRequest) -> XtbPathResult:
    """在持久运行目录里跑一次 MTD 路径搜索。

    同步函数（子进程 + 文件 IO）；异步调用方必须经 ``asyncio.to_thread`` 卸载。
    """
    try:
        reactant_symbols, _ = _symbols_and_positions(request.reactant_xyz, label="reactant")
        product_symbols, _ = _symbols_and_positions(request.product_xyz, label="product")
    except ValueError as exc:
        # 端点先自检：几何不完整或元素序列不一致时，跑引擎只会浪费机时
        output_dir = request.output_dir
        os.makedirs(output_dir, exist_ok=True)
        run_dir = _next_run_dir(output_dir, request.run_label)
        return XtbPathResult(
            run_dir=run_dir, termination_reason="invalid_endpoints",
            message=f"the endpoints were rejected before starting xtb: {exc}",
            diagnostics=[str(exc)],
        )

    command = build_xtb_path_command(request)
    run_dir = _next_run_dir(request.output_dir, request.run_label)
    reactant_path = os.path.join(run_dir, "reactant.xyz")
    product_path = os.path.join(run_dir, "product.xyz")
    command_path = os.path.join(run_dir, "command.txt")
    path_input_path = os.path.join(run_dir, PATH_INPUT_NAME)
    versions_path = os.path.join(run_dir, "versions.txt")
    stdout_path = os.path.join(run_dir, "stdout.log")
```

```
cloudra/tools/core/xtb_path_runner.py:394       stderr_path = os.path.join(run_dir, "stderr.log")
cloudra/tools/core/xtb_path_runner.py:395
cloudra/tools/core/xtb_path_runner.py:396       atomic_write_text(reactant_path, request.reactant_xyz)
cloudra/tools/core/xtb_path_runner.py:397       atomic_write_text(product_path, request.product_xyz)
cloudra/tools/core/xtb_path_runner.py:398       atomic_write_text(command_path, " ".join(command) + "\n")
cloudra/tools/core/xtb_path_runner.py:399       atomic_write_text(path_input_path, build_path_input(request.refinement_runs
))
cloudra/tools/core/xtb_path_runner.py:400       atomic_write_text(versions_path, _capture_versions())
cloudra/tools/core/xtb_path_runner.py:401
cloudra/tools/core/xtb_path_runner.py:402       result = XtbPathResult(
cloudra/tools/core/xtb_path_runner.py:403           run_dir=run_dir, command=command, symbols=reactant_symbols,
cloudra/tools/core/xtb_path_runner.py:404           reactant_path=reactant_path, product_path=product_path,
cloudra/tools/core/xtb_path_runner.py:405           command_path=command_path, path_input_path=path_input_path,
versions_path=versions_path,
cloudra/tools/core/xtb_path_runner.py:406           refinement_runs=request.refinement_runs,
cloudra/tools/core/xtb_path_runner.py:407           stdout_path=stdout_path, stderr_path=stderr_path,
cloudra/tools/core/xtb_path_runner.py:408           path_xyz_path=os.path.join(run_dir, PATH_XYZ_NAME),
cloudra/tools/core/xtb_path_runner.py:409           ts_xyz_path=os.path.join(run_dir, TS_XYZ_NAME),
cloudra/tools/core/xtb_path_runner.py:410       )
cloudra/tools/core/xtb_path_runner.py:411
cloudra/tools/core/xtb_path_runner.py:412       if reactant_symbols != product_symbols:
cloudra/tools/core/xtb_path_runner.py:413           result.termination_reason = "invalid_endpoints"
cloudra/tools/core/xtb_path_runner.py:414           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:415               f"reactant and product element sequences differ ({reactant_symbols}
 vs "
cloudra/tools/core/xtb_path_runner.py:416               f"{product_symbols}); the path search needs one consistent atom set
"
cloudra/tools/core/xtb_path_runner.py:417           )
cloudra/tools/core/xtb_path_runner.py:418           result.message = "Endpoints rejected: the element sequences differ."
cloudra/tools/core/xtb_path_runner.py:419           result.artifacts = _list_artifacts(run_dir)
cloudra/tools/core/xtb_path_runner.py:420           return result
cloudra/tools/core/xtb_path_runner.py:421
cloudra/tools/core/xtb_path_runner.py:422       import shutil
cloudra/tools/core/xtb_path_runner.py:423
cloudra/tools/core/xtb_path_runner.py:424       if not shutil.which("xtb"):
cloudra/tools/core/xtb_path_runner.py:425           result.termination_reason = "executable_missing"
cloudra/tools/core/xtb_path_runner.py:426           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:427               "the 'xtb' executable was not found on PATH; no run was started.
The caller should "
cloudra/tools/core/xtb_path_runner.py:428               f"fall back to IDPP interpolation and report path_kind={
PATH_KIND_IDPP!r}"
cloudra/tools/core/xtb_path_runner.py:429           )
cloudra/tools/core/xtb_path_runner.py:430           atomic_write_text(stdout_path, "")
cloudra/tools/core/xtb_path_runner.py:431           atomic_write_text(stderr_path, "")
cloudra/tools/core/xtb_path_runner.py:432           result.artifacts = _list_artifacts(run_dir)
cloudra/tools/core/xtb_path_runner.py:433           result.message = "xtb executable not found; no path search was started
."
cloudra/tools/core/xtb_path_runner.py:434           return result
cloudra/tools/core/xtb_path_runner.py:435
cloudra/tools/core/xtb_path_runner.py:436       timed_out = False
cloudra/tools/core/xtb_path_runner.py:437       proc = subprocess.Popen(
cloudra/tools/core/xtb_path_runner.py:438           command, cwd=run_dir, stdout=subprocess.PIPE, stderr=subprocess.PIPE,
cloudra/tools/core/xtb_path_runner.py:439           text=True, start_new_session=True,
cloudra/tools/core/xtb_path_runner.py:440       )
cloudra/tools/core/xtb_path_runner.py:441       try:
cloudra/tools/core/xtb_path_runner.py:442           stdout, stderr = proc.communicate(timeout=request.timeout)
cloudra/tools/core/xtb_path_runner.py:443           returncode = proc.returncode
cloudra/tools/core/xtb_path_runner.py:444       except subprocess.TimeoutExpired:
cloudra/tools/core/xtb_path_runner.py:445           timed_out = True
```

```
cloudra/tools/core/xtb_path_runner.py:446            _terminate_process_group(proc)
cloudra/tools/core/xtb_path_runner.py:447            stdout, stderr = proc.communicate()
cloudra/tools/core/xtb_path_runner.py:448            returncode = proc.returncode
cloudra/tools/core/xtb_path_runner.py:449            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:450                f"the xtb path search exceeded the {request.timeout}s timeout; this run's own process "
cloudra/tools/core/xtb_path_runner.py:451                "group was terminated (its logs and intermediate files are kept)"
cloudra/tools/core/xtb_path_runner.py:452            )
cloudra/tools/core/xtb_path_runner.py:453
cloudra/tools/core/xtb_path_runner.py:454        atomic_write_text(stdout_path, stdout or "")
cloudra/tools/core/xtb_path_runner.py:455        atomic_write_text(stderr_path, stderr or "")
cloudra/tools/core/xtb_path_runner.py:456        result.returncode = returncode
cloudra/tools/core/xtb_path_runner.py:457        result.artifacts = _list_artifacts(run_dir)
cloudra/tools/core/xtb_path_runner.py:458
cloudra/tools/core/xtb_path_runner.py:459        # 实测（本机 xtb 6.7.1）：``normal termination of xtb`` 落在 **stderr**，而计时块落在
cloudra/tools/core/xtb_path_runner.py:460        # stdout。因此标记必须在两个流上一起扫——只看 stdout 会把正常的运行判成失败。
cloudra/tools/core/xtb_path_runner.py:461        markers = _scan_markers(
cloudra/tools/core/xtb_path_runner.py:462            (stdout or "").splitlines() + (stderr or "").splitlines()
cloudra/tools/core/xtb_path_runner.py:463        )
cloudra/tools/core/xtb_path_runner.py:464        result.evidence = markers.evidence
cloudra/tools/core/xtb_path_runner.py:465        result.refinement_termination = markers.refinement_termination
cloudra/tools/core/xtb_path_runner.py:466
cloudra/tools/core/xtb_path_runner.py:467        if timed_out:
cloudra/tools/core/xtb_path_runner.py:468            result.termination_reason = "timeout"
cloudra/tools/core/xtb_path_runner.py:469            result.message = (
cloudra/tools/core/xtb_path_runner.py:470                f"the xtb path search timed out after {request.timeout}s (returncode={returncode}). "
cloudra/tools/core/xtb_path_runner.py:471                f"Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:472            )
cloudra/tools/core/xtb_path_runner.py:473            return result
cloudra/tools/core/xtb_path_runner.py:474
cloudra/tools/core/xtb_path_runner.py:475        if returncode != 0:
cloudra/tools/core/xtb_path_runner.py:476            result.termination_reason = "failed"
cloudra/tools/core/xtb_path_runner.py:477            tail = "\n".join((stdout or "").splitlines()[-15:]) or (stderr or "")[-2000:]
cloudra/tools/core/xtb_path_runner.py:478            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:479                "xtb exited non-zero; raw log tail follows (no cause is guessed)"
cloudra/tools/core/xtb_path_runner.py:480            )
cloudra/tools/core/xtb_path_runner.py:481            result.diagnostics.append(tail)
cloudra/tools/core/xtb_path_runner.py:482            result.message = (
cloudra/tools/core/xtb_path_runner.py:483                f"the xtb path search failed (returncode={returncode}). Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:484            )
cloudra/tools/core/xtb_path_runner.py:485            return result
cloudra/tools/core/xtb_path_runner.py:486
cloudra/tools/core/xtb_path_runner.py:487        if not markers.normal_termination:
cloudra/tools/core/xtb_path_runner.py:488            result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:489            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:490                f"xtb exited 0 but never printed its normal-termination marker "
cloudra/tools/core/xtb_path_runner.py:491                f"({MARKER_NORMAL_TERMINATION!r}, which this build writes to stderr); refusing to treat "
cloudra/tools/core/xtb_path_runner.py:492                "the exit code alone as success"
cloudra/tools/core/xtb_path_runner.py:493            )
cloudra/tools/core/xtb_path_runner.py:494            result.message = f"xtb did not report normal termination. Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:495            return result
cloudra/tools/core/xtb_path_runner.py:496
cloudra/tools/core/xtb_path_runner.py:497        if not markers.path_finder:
```

```
cloudra/tools/core/xtb_path_runner.py:498           result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:499           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:500               f"xtb exited 0 but never printed its path-finder timing block "
cloudra/tools/core/xtb_path_runner.py:501               f"({MARKER_PATH_FINDER.strip()!r}); refusing to treat the exit code alone as success"
cloudra/tools/core/xtb_path_runner.py:502           )
cloudra/tools/core/xtb_path_runner.py:503           result.message = f"xtb did not report a completed path search. Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:504           return result
cloudra/tools/core/xtb_path_runner.py:505
cloudra/tools/core/xtb_path_runner.py:506       # 产物完整性：路径与 TS 候选都要在，且几何自洽——"文件在"不构成成功判据
cloudra/tools/core/xtb_path_runner.py:507       try:
cloudra/tools/core/xtb_path_runner.py:508           frames = _parse_path_frames(result.path_xyz_path)
cloudra/tools/core/xtb_path_runner.py:509           ts_energy, ts_positions = _parse_ts_candidate(result.ts_xyz_path)
cloudra/tools/core/xtb_path_runner.py:510       except (OSError, ValueError) as exc:
cloudra/tools/core/xtb_path_runner.py:511           result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:512           result.diagnostics.append(f"path artifacts are unusable: {exc}")
cloudra/tools/core/xtb_path_runner.py:513           result.message = (
cloudra/tools/core/xtb_path_runner.py:514               f"xtb reported normal termination but its path artifacts are unusable ({exc}). "
cloudra/tools/core/xtb_path_runner.py:515               f"Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:516           )
cloudra/tools/core/xtb_path_runner.py:517           return result
cloudra/tools/core/xtb_path_runner.py:518
cloudra/tools/core/xtb_path_runner.py:519       for frame in frames:
cloudra/tools/core/xtb_path_runner.py:520           if len(frame.positions) != len(reactant_symbols):
cloudra/tools/core/xtb_path_runner.py:521               result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:522               result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:523                   f"frame {frame.frame_index} has {len(frame.positions)} atoms but the endpoints have "
cloudra/tools/core/xtb_path_runner.py:524                   f"{len(reactant_symbols)}"
cloudra/tools/core/xtb_path_runner.py:525               )
cloudra/tools/core/xtb_path_runner.py:526               result.message = "xtb reported normal termination but the path frames are inconsistent."
cloudra/tools/core/xtb_path_runner.py:527               return result
cloudra/tools/core/xtb_path_runner.py:528       if len(frames) < 2:
cloudra/tools/core/xtb_path_runner.py:529           result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:530           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:531               f"the path has only {len(frames)} frame(s); a path needs at least the two endpoints"
cloudra/tools/core/xtb_path_runner.py:532           )
cloudra/tools/core/xtb_path_runner.py:533           result.message = "xtb reported normal termination but produced no usable path."
cloudra/tools/core/xtb_path_runner.py:534           return result
cloudra/tools/core/xtb_path_runner.py:535       if len(ts_positions) != len(reactant_symbols):
cloudra/tools/core/xtb_path_runner.py:536           result.termination_reason = "incomplete_artifacts"
cloudra/tools/core/xtb_path_runner.py:537           result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:538               f"the TS candidate has {len(ts_positions)} atoms but the endpoints have "
cloudra/tools/core/xtb_path_runner.py:539               f"{len(reactant_symbols)}"
cloudra/tools/core/xtb_path_runner.py:540           )
cloudra/tools/core/xtb_path_runner.py:541           result.message = "xtb reported normal termination but the TS candidate is inconsistent."
cloudra/tools/core/xtb_path_runner.py:542           return result
cloudra/tools/core/xtb_path_runner.py:543
cloudra/tools/core/xtb_path_runner.py:544       result.frames = frames
cloudra/tools/core/xtb_path_runner.py:545       result.ts_candidate_energy_eh = ts_energy
cloudra/tools/core/xtb_path_runner.py:546       result.ts_candidate_positions = ts_positions
cloudra/tools/core/xtb_path_runner.py:547
cloudra/tools/core/xtb_path_runner.py:548       if markers.refinement_termination == "cycles_exhausted":
cloudra/tools/core/xtb_path_runner.py:549           # **被截断的路径不是初猜**：实测这种路径的最高帧落在反应物自己的盆地里
```

```
cloudra/tools/core/xtb_path_runner.py:550            # （离起点只有 21°、相对能 44.865 kcal/mol，做 OptTS 跑 56 轮不收敛）。
cloudra/tools/core/xtb_path_runner.py:551            # 这里把截断的现场（路径帧 + TS 候选）保留下来供复查，但明确标成不可用。
cloudra/tools/core/xtb_path_runner.py:552            result.termination_reason = "path_not_converged"
cloudra/tools/core/xtb_path_runner.py:553            result.usable_as_guess = False
cloudra/tools/core/xtb_path_runner.py:554            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:555                f"the engine stopped its path refinement at the {request.refinement_runs}-run cap "
cloudra/tools/core/xtb_path_runner.py:556                f"({MARKER_PATH_CYCLES_EXHAUSTED!r}); that is a truncated path, not a converged one. "
cloudra/tools/core/xtb_path_runner.py:557                "The frames and TS candidate are kept for inspection only. Measured consequence of "
cloudra/tools/core/xtb_path_runner.py:558                "accepting a truncated path (docs/evidence/ts_guess_mtd_artifact/): its highest frame "
cloudra/tools/core/xtb_path_runner.py:559                "sits in the reactant's own basin, 21 deg from the start, and an ORCA OptTS started "
cloudra/tools/core/xtb_path_runner.py:560                "from it never converged (56 cycles, slid back to the minimum). Raise "
cloudra/tools/core/xtb_path_runner.py:561                "`refinement_runs` and rerun"
cloudra/tools/core/xtb_path_runner.py:562            )
cloudra/tools/core/xtb_path_runner.py:563            result.message = (
cloudra/tools/core/xtb_path_runner.py:564                f"the xTB path refinement hit its {request.refinement_runs}-run cap before the engine "
cloudra/tools/core/xtb_path_runner.py:565                f"declared convergence, so no usable TS guess was produced. Raise `refinement_runs` and "
cloudra/tools/core/xtb_path_runner.py:566                f"rerun. Run directory (truncated path kept for inspection): {run_dir}"
cloudra/tools/core/xtb_path_runner.py:567            )
cloudra/tools/core/xtb_path_runner.py:568            return result
cloudra/tools/core/xtb_path_runner.py:569
cloudra/tools/core/xtb_path_runner.py:570        if markers.refinement_termination != "converged":
cloudra/tools/core/xtb_path_runner.py:571            result.termination_reason = "unknown_failure"
cloudra/tools/core/xtb_path_runner.py:572            result.usable_as_guess = False
cloudra/tools/core/xtb_path_runner.py:573            result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:574                "xtb exited 0 and produced path artifacts, but its log carries neither refinement "
cloudra/tools/core/xtb_path_runner.py:575                f"termination line ({MARKER_PATH_CONVERGED!r} / {MARKER_PATH_CYCLES_EXHAUSTED!r}); "
cloudra/tools/core/xtb_path_runner.py:576                "whether the path refinement converged cannot be established from this output, and a "
cloudra/tools/core/xtb_path_runner.py:577                "non-converged path must not be handed over as a TS guess"
cloudra/tools/core/xtb_path_runner.py:578            )
cloudra/tools/core/xtb_path_runner.py:579            result.message = (
cloudra/tools/core/xtb_path_runner.py:580                f"the xTB path search reported no path-refinement termination line, so convergence "
cloudra/tools/core/xtb_path_runner.py:581                f"could not be confirmed. Full log: {stdout_path}"
cloudra/tools/core/xtb_path_runner.py:582            )
cloudra/tools/core/xtb_path_runner.py:583            return result
cloudra/tools/core/xtb_path_runner.py:584
cloudra/tools/core/xtb_path_runner.py:585        result.termination_reason = "completed"
cloudra/tools/core/xtb_path_runner.py:586        result.usable_as_guess = True
cloudra/tools/core/xtb_path_runner.py:587        result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:588            "path-frame comment energies are RELATIVE kcal/mol; the TS candidate's comment energy is "
cloudra/tools/core/xtb_path_runner.py:589            "ABSOLUTE Eh — the two are reported in separate fields on purpose "
cloudra/tools/core/xtb_path_runner.py:590            "(verified against single points on this machine)"
cloudra/tools/core/xtb_path_runner.py:591        )
cloudra/tools/core/xtb_path_runner.py:592        result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:593            "the TS candidate is a guess sampled by metadynamics, NOT a saddle point and NOT verified; "
cloudra/tools/core/xtb_path_runner.py:594            "frequency/IRC judgement still has to run"
cloudra/tools/core/xtb_path_runner.py:595        )
cloudra/tools/core/xtb_path_runner.py:596        result.diagnostics.append(
cloudra/tools/core/xtb_path_runner.py:597            f"the engine declared the path refinement converged ({MARKER_PATH_CONVERGED!r}) within the "
cloudra/tools/core/xtb_path_runner.py:598            f"{request.refinement_runs}-run cap"
cloudra/tools/core/xtb_path_runner.py:599        )
cloudra/tools/core/xtb_path_runner.py:600        result.message = (
cloudra/tools/core/xtb_path_runner.py:601            f"xtb path search completed: {len(frames)} path frames and a TS candidate "
```

```
cloudra/tools/core/xtb_path_runner.py:602           f"({ts_energy:.6f} Eh). Run directory: {run_dir}"
cloudra/tools/core/xtb_path_runner.py:603       )
cloudra/tools/core/xtb_path_runner.py:604       return result
cloudra/tools/core/xtb_path_runner.py:605
cloudra/tools/core/xtb_path_runner.py:606
cloudra/tools/core/xtb_path_runner.py:607   def _terminate_process_group(proc: subprocess.Popen) -> None:
cloudra/tools/core/xtb_path_runner.py:608       """先 SIGTERM 再 SIGKILL **本进程组**，并 wait/reap，绝不只发信号不回收。"""
cloudra/tools/core/xtb_path_runner.py:609       try:
cloudra/tools/core/xtb_path_runner.py:610           pgid = os.getpgid(proc.pid)
cloudra/tools/core/xtb_path_runner.py:611       except ProcessLookupError:
cloudra/tools/core/xtb_path_runner.py:612           proc.wait()
cloudra/tools/core/xtb_path_runner.py:613           return
cloudra/tools/core/xtb_path_runner.py:614       for sig in (signal.SIGTERM, signal.SIGKILL):
cloudra/tools/core/xtb_path_runner.py:615           try:
cloudra/tools/core/xtb_path_runner.py:616               os.killpg(pgid, sig)
cloudra/tools/core/xtb_path_runner.py:617           except ProcessLookupError:
cloudra/tools/core/xtb_path_runner.py:618               break
cloudra/tools/core/xtb_path_runner.py:619           try:
cloudra/tools/core/xtb_path_runner.py:620               proc.wait(timeout=10)
cloudra/tools/core/xtb_path_runner.py:621               return
cloudra/tools/core/xtb_path_runner.py:622           except subprocess.TimeoutExpired:
cloudra/tools/core/xtb_path_runner.py:623               continue
cloudra/tools/core/xtb_path_runner.py:624       proc.wait()
cloudra/tools/name_hints.py:1   """Retired wire names → hint only. Never executed."""
cloudra/tools/name_hints.py:2
cloudra/tools/name_hints.py:3   _RETIRED: dict[str, str] = {
cloudra/tools/name_hints.py:4       "text_reader": "read",
cloudra/tools/name_hints.py:5       "image_reader": "read",
cloudra/tools/name_hints.py:6       "pdf2md": "read",
cloudra/tools/name_hints.py:7       "pdf2image": "read",
cloudra/tools/name_hints.py:8       "read_file": "read",
cloudra/tools/name_hints.py:9       "web_search": "search (source=web)",
cloudra/tools/name_hints.py:10      "paper_searcher": "search (source=arxiv)",
cloudra/tools/name_hints.py:11      "openalex_searcher": "search (source=openalex)",
cloudra/tools/name_hints.py:12      "bashtools": "bash",
cloudra/tools/name_hints.py:13      "shell": "bash",
cloudra/tools/name_hints.py:14      "text_writer": "write",
cloudra/tools/name_hints.py:15      "write_file": "write",
cloudra/tools/name_hints.py:16      "edit_file": "edit",
cloudra/tools/name_hints.py:17      "str_replace": "edit",
cloudra/tools/name_hints.py:18      "getskill": "get_skill",
cloudra/tools/name_hints.py:19      "web_fetcher": "web_fetch",
cloudra/tools/name_hints.py:20      "headless_browser": "browser",
cloudra/tools/name_hints.py:21      "SubAgentLauncher": "subagent_launcher",
cloudra/tools/name_hints.py:22      "SubAgentSwarm": "subagent_swarm",
cloudra/tools/name_hints.py:23  }
cloudra/tools/name_hints.py:24
cloudra/tools/name_hints.py:25
cloudra/tools/name_hints.py:26  def hint_unknown_tool(name: str) -> str:
cloudra/tools/name_hints.py:27      target = _RETIRED.get(name)
cloudra/tools/name_hints.py:28      if target:
cloudra/tools/name_hints.py:29          return (
```

```
cloudra/tools/name_hints.py:30              f"[Tool error] unknown tool {name!r}. "
cloudra/tools/name_hints.py:31              f"Use `{target}` instead."
cloudra/tools/name_hints.py:32          )
cloudra/tools/name_hints.py:33      return (
cloudra/tools/name_hints.py:34          f"[Tool error] unknown tool {name!r}. "
cloudra/tools/name_hints.py:35          "Choose an available tool and retry."
cloudra/tools/name_hints.py:36      )
cloudra/tools/name_hints.py:37
cloudra/tools/name_hints.py:38
cloudra/tools/name_hints.py:39  def is_retired_tool_name(name: str) -> bool:
cloudra/tools/name_hints.py:40      return name in _RETIRED
cloudra/tools/registry.py:1 """Explicit built-in tool manifest.
cloudra/tools/registry.py:2
cloudra/tools/registry.py:3 Keep this module dependency-free: :mod:`cloudra.core.agent` imports
cloudra/tools/registry.py:4 ``cloudra.tools`` while several tool implementations import ``cloudra.core.agent``.
cloudra/tools/registry.py:5 The registry therefore stores import paths instead of importing tool classes at
cloudra/tools/registry.py:6 module import time.
cloudra/tools/registry.py:7
cloudra/tools/registry.py:8 Optional semantic fields are the single source of truth for downstream policy
cloudra/tools/registry.py:9 projections (DAG gate / runtime profiles / evidence derivation).  Downstream
cloudra/tools/registry.py:10    modules must query :mod:`cloudra.tools.capabilities` instead of maintaining
cloudra/tools/registry.py:11    their own per-tool name sets.
cloudra/tools/registry.py:12    """
cloudra/tools/registry.py:13
cloudra/tools/registry.py:14    from dataclasses import dataclass
cloudra/tools/registry.py:15
cloudra/tools/registry.py:16
cloudra/tools/registry.py:17    @dataclass(frozen=True)
cloudra/tools/registry.py:18    class ToolRegistration:
cloudra/tools/registry.py:19        module: str
cloudra/tools/registry.py:20        class_name: str
cloudra/tools/registry.py:21
cloudra/tools/registry.py:22        # Declared wire name. Only needed when the tool has non-default semantics
cloudra/tools/registry.py:23        # (or a tool_name that differs from class_name); capabilities only projects
cloudra/tools/registry.py:24        # entries that carry it.
cloudra/tools/registry.py:25        tool_name: str = ""
cloudra/tools/registry.py:26
cloudra/tools/registry.py:27        # DAG semantics. All flags default to the conservative/plain behavior.
cloudra/tools/registry.py:28        dag_observe: bool = True
cloudra/tools/registry.py:29        evidence_deriver: str = ""  # "module:function(current, role, arguments, result)->dict"
cloudra/tools/registry.py:30        # 事实适配器（contract_version=2）："module:function(payload, source_ref)
cloudra/tools/registry.py:31        # -> list[EvidenceFact]"：从已登记证据中抽类型化事实（含反证）。
cloudra/tools/registry.py:32        dag_fact_adapter: str = ""
cloudra/tools/registry.py:33        # 证据产生工具：完成的调用产出计算证据（知识检索触发语义）。
cloudra/tools/registry.py:34        # §5 起不再有"自动入 CallReview 收件箱"的开关——送审只能显式
cloudra/tools/registry.py:35        # （dag_bind_evidence with claim）。
cloudra/tools/registry.py:36        dag_evidence_producer: bool = False
cloudra/tools/registry.py:37        # Phase 3 门槛：mechanism 未被接受前拦截的计算入口（开算/写计算输入）。
cloudra/tools/registry.py:38        dag_compute_entry: bool = False
cloudra/tools/registry.py:39        dag_compute_predicate: str = ""  # "module:function(arguments)->bool", overrides
dag_compute_entry
cloudra/tools/registry.py:40
cloudra/tools/registry.py:41        # Runtime availability. The default keeps the Android allow-list opt-in.
```

```
cloudra/tools/registry.py:42        runtime_profiles: frozenset[str] = frozenset({"desktop"})
cloudra/tools/registry.py:43
cloudra/tools/registry.py:44        # Visual-mode gate: image-producing/consuming tools hidden unless the
cloudra/tools/registry.py:45        # agent's visual_mode setting is on.
cloudra/tools/registry.py:46        visual: bool = False
cloudra/tools/registry.py:47
cloudra/tools/registry.py:48        # Supervisor capability: whether the scientific Supervisor may discover and
cloudra/tools/registry.py:49        # execute this tool. One of: "none" (default, hidden), "evidence_read"
cloudra/tools/registry.py:50        # (read-only evidence access), "skill_read", "hub" (AgentHub, action-gated),
cloudra/tools/registry.py:51        # "review_decision" (control-plane verdicts). Projected exclusively via
cloudra/tools/registry.py:52        # tools/capabilities.py; do not keep separate supervisor tool name sets.
cloudra/tools/registry.py:53        supervisor_access: str = "none"
cloudra/tools/registry.py:54
cloudra/tools/registry.py:55        # Invocation routing. Direct controls stay top-level wire tools in the
cloudra/tools/registry.py:56        # javascript protocol (exposed next to js_exec) and are rejected inside a
cloudra/tools/registry.py:57        # guest program; ordinary business tools are the only guest-callable ones.
cloudra/tools/registry.py:58        invocation_direct_only: bool = False
cloudra/tools/registry.py:59
cloudra/tools/registry.py:60        # Config gate: "<section>.<field>" naming a boolean on CloudraConfig.
cloudra/tools/registry.py:61        # When the flag is false the tool is hidden from the wire projection and
cloudra/tools/registry.py:62        # hard-fails at call time (the tool itself must check the flag too).
cloudra/tools/registry.py:63        config_gate: str = ""
cloudra/tools/registry.py:64
cloudra/tools/registry.py:65
cloudra/tools/registry.py:66    TOOL_REGISTRY: tuple[ToolRegistration, ...] = (
cloudra/tools/registry.py:67        ToolRegistration(
cloudra/tools/registry.py:68            module='cloudra.tools.Bashtool.bash',
cloudra/tools/registry.py:69            class_name='Bash',
cloudra/tools/registry.py:70            tool_name='bash',
cloudra/tools/registry.py:71            dag_compute_predicate='cloudra.tools.core.bash_habit_guard:
is_substantive_bash_command',
cloudra/tools/registry.py:72        ),
cloudra/tools/registry.py:73        ToolRegistration(
cloudra/tools/registry.py:74            module='cloudra.tools.Browsertool.browser',
cloudra/tools/registry.py:75            class_name='HeadlessBrowser',
cloudra/tools/registry.py:76            tool_name='browser',
cloudra/tools/registry.py:77        ),
cloudra/tools/registry.py:78        ToolRegistration(
cloudra/tools/registry.py:79            module='cloudra.tools.Browsertool.web_fetch',
cloudra/tools/registry.py:80            class_name='WebFetcher',
cloudra/tools/registry.py:81            tool_name='web_fetch',
cloudra/tools/registry.py:82            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:83        ),
cloudra/tools/registry.py:84        ToolRegistration(
cloudra/tools/registry.py:85            module='cloudra.tools.Chemtool.conformer_search',
cloudra/tools/registry.py:86            class_name='ConformerSearch',
cloudra/tools/registry.py:87            tool_name='conformer_search',
cloudra/tools/registry.py:88            dag_compute_entry=True,
cloudra/tools/registry.py:89        ),
cloudra/tools/registry.py:90        ToolRegistration(
cloudra/tools/registry.py:91            module='cloudra.tools.Chemtool.electronic_analysis',
cloudra/tools/registry.py:92            class_name='ElectronicAnalysis',
cloudra/tools/registry.py:93            tool_name='electronic_analysis',
```

```
cloudra/tools/registry.py:94            dag_compute_entry=True,
cloudra/tools/registry.py:95        ),
cloudra/tools/registry.py:96        ToolRegistration('cloudra.tools.Chemtool.format_converter', 'FormatConverter'),
cloudra/tools/registry.py:97        ToolRegistration(
cloudra/tools/registry.py:98            module='cloudra.tools.Chemtool.qm_input_writer',
cloudra/tools/registry.py:99            class_name='QmInputWriter',
cloudra/tools/registry.py:100           tool_name='qm_input_writer',
cloudra/tools/registry.py:101           dag_compute_entry=True,
cloudra/tools/registry.py:102       ),
cloudra/tools/registry.py:103       ToolRegistration('cloudra.tools.Chemtool.image_to_smiles', 'ImageToSmiles'),
cloudra/tools/registry.py:104       ToolRegistration(
cloudra/tools/registry.py:105           module='cloudra.tools.Chemtool.cdxml_to_graph',
cloudra/tools/registry.py:106           class_name='CdxmlToGraph',
cloudra/tools/registry.py:107           tool_name='cdxml_to_graph',
cloudra/tools/registry.py:108       ),
cloudra/tools/registry.py:109       ToolRegistration('cloudra.tools.Chemtool.internal_coordinates', 'InternalCoordinates'),
cloudra/tools/registry.py:110       ToolRegistration(
cloudra/tools/registry.py:111           module='cloudra.tools.Chemtool.ir_processor',
cloudra/tools/registry.py:112           class_name='IRProcessor',
cloudra/tools/registry.py:113           tool_name='ir_processor',
cloudra/tools/registry.py:114       ),
cloudra/tools/registry.py:115       ToolRegistration(
cloudra/tools/registry.py:116           module='cloudra.tools.Chemtool.log_parser',
cloudra/tools/registry.py:117           class_name='LogParser',
cloudra/tools/registry.py:118           tool_name='log_parser',
cloudra/tools/registry.py:119           evidence_deriver='cloudra.tools.Chemtool.log_parser:derive_delivery_changes',
cloudra/tools/registry.py:120           dag_fact_adapter='cloudra.dag.fact_adapters:log_parser_facts',
cloudra/tools/registry.py:121           dag_evidence_producer=True,
cloudra/tools/registry.py:122           supervisor_access='evidence_read',
cloudra/tools/registry.py:123       ),
cloudra/tools/registry.py:124       ToolRegistration('cloudra.tools.Chemtool.molecule_drawer', 'MoleculeDrawer'),
cloudra/tools/registry.py:125       ToolRegistration(
cloudra/tools/registry.py:126           module='cloudra.tools.Chemtool.mechanism_predict',
cloudra/tools/registry.py:127           class_name='MechanismPredict',
cloudra/tools/registry.py:128           tool_name='mechanism_predict',
cloudra/tools/registry.py:129           # ML 机理假设生成器（FlowER 外部 conda env）：不是计算证据，默认
cloudra/tools/registry.py:130           # 由 flower.enabled 开关关闭。
cloudra/tools/registry.py:131           config_gate='flower.enabled',
cloudra/tools/registry.py:132       ),
cloudra/tools/registry.py:133       ToolRegistration('cloudra.tools.Chemtool.name_to_structure', 'NameToStructure'),
cloudra/tools/registry.py:134       ToolRegistration(
cloudra/tools/registry.py:135           module='cloudra.tools.Chemtool.nmr_processor',
cloudra/tools/registry.py:136           class_name='NMRProcessor',
cloudra/tools/registry.py:137           tool_name='nmr_processor',
cloudra/tools/registry.py:138       ),
cloudra/tools/registry.py:139       ToolRegistration(
cloudra/tools/registry.py:140           module='cloudra.tools.Chemtool.pymol_render',
cloudra/tools/registry.py:141           class_name='PymolRender',
cloudra/tools/registry.py:142           tool_name='pymol_render',
cloudra/tools/registry.py:143           visual=True,
cloudra/tools/registry.py:144       ),
cloudra/tools/registry.py:145       ToolRegistration('cloudra.tools.Chemtool.query_compute_job', 'QueryComputeJob'),
```

```
cloudra/tools/registry.py:146       ToolRegistration('cloudra.tools.Chemtool.query_compute_queue', 'QueryComputeQueue'),
cloudra/tools/registry.py:147       ToolRegistration(
cloudra/tools/registry.py:148           module='cloudra.tools.Chemtool.mark_compute_job_failed',
cloudra/tools/registry.py:149           class_name='MarkComputeJobFailed',
cloudra/tools/registry.py:150           tool_name='mark_compute_job_failed',
cloudra/tools/registry.py:151           dag_observe=False,
cloudra/tools/registry.py:152       ),
cloudra/tools/registry.py:153       ToolRegistration(
cloudra/tools/registry.py:154           module='cloudra.tools.Chemtool.reaction_mapping',
cloudra/tools/registry.py:155           class_name='ReactionMapping',
cloudra/tools/registry.py:156           tool_name='reaction_mapping',
cloudra/tools/registry.py:157           evidence_deriver='cloudra.tools.Chemtool.reaction_mapping:derive_delivery_changes',
cloudra/tools/registry.py:158           dag_fact_adapter='cloudra.dag.fact_adapters:reaction_mapping_facts',
cloudra/tools/registry.py:159           dag_evidence_producer=True,
cloudra/tools/registry.py:160           supervisor_access='evidence_read',
cloudra/tools/registry.py:161       ),
cloudra/tools/registry.py:162       # TS 搜索 SOP 的 agent 可见接口（只有这两个函数：刻意没有 status/查询接口）
cloudra/tools/registry.py:163       ToolRegistration(
cloudra/tools/registry.py:164           'cloudra.tools.Chemtool.ts_search_process', 'TsPrepare', tool_name='ts_prepare',
cloudra/tools/registry.py:165       ),
cloudra/tools/registry.py:166       ToolRegistration(
cloudra/tools/registry.py:167           'cloudra.tools.Chemtool.ts_search_process', 'TsResume', tool_name='ts_resume',
cloudra/tools/registry.py:168       ),
cloudra/tools/registry.py:169       ToolRegistration('cloudra.tools.Chemtool.smiles_to_mol', 'SmilesToMol'),
cloudra/tools/registry.py:170       ToolRegistration(
cloudra/tools/registry.py:171           module='cloudra.tools.Chemtool.submit_compute_job',
cloudra/tools/registry.py:172           class_name='SubmitComputeJob',
cloudra/tools/registry.py:173           tool_name='submit_compute_job',
cloudra/tools/registry.py:174           dag_compute_entry=True,
cloudra/tools/registry.py:175       ),
cloudra/tools/registry.py:176       ToolRegistration(
cloudra/tools/registry.py:177           module='cloudra.tools.Chemtool.thermo_analysis',
cloudra/tools/registry.py:178           class_name='ThermoAnalysis',
cloudra/tools/registry.py:179           tool_name='thermo_analysis',
cloudra/tools/registry.py:180           evidence_deriver='cloudra.tools.Chemtool.thermo_analysis:derive_delivery_changes',
cloudra/tools/registry.py:181           dag_evidence_producer=True,
cloudra/tools/registry.py:182       ),
cloudra/tools/registry.py:183       ToolRegistration(
cloudra/tools/registry.py:184           module='cloudra.tools.Chemtool.ts_guess',
cloudra/tools/registry.py:185           class_name='TsGuess',
cloudra/tools/registry.py:186           tool_name='ts_guess',
cloudra/tools/registry.py:187           evidence_deriver='cloudra.tools.Chemtool.ts_guess:derive_delivery_changes',
cloudra/tools/registry.py:188           dag_fact_adapter='cloudra.dag.fact_adapters:ts_guess_facts',
cloudra/tools/registry.py:189           dag_evidence_producer=True,
cloudra/tools/registry.py:190           dag_compute_predicate='cloudra.tools.Chemtool.ts_guess:is_compute_entry_call',
cloudra/tools/registry.py:191       ),
cloudra/tools/registry.py:192       ToolRegistration(
cloudra/tools/registry.py:193           module='cloudra.tools.Interactool.ask_user',
cloudra/tools/registry.py:194           class_name='AskUser',
cloudra/tools/registry.py:195           tool_name='ask_user',
cloudra/tools/registry.py:196           invocation_direct_only=True,
cloudra/tools/registry.py:197       ),
```

```
cloudra/tools/registry.py:198       ToolRegistration('cloudra.tools.Interactool.email_reader', 'EmailReader'),
cloudra/tools/registry.py:199       ToolRegistration('cloudra.tools.Interactool.email_sender', 'EmailSender'),
cloudra/tools/registry.py:200       ToolRegistration(
cloudra/tools/registry.py:201           module='cloudra.tools.Managetool.agent_hub',
cloudra/tools/registry.py:202           class_name='AgentHub',
cloudra/tools/registry.py:203           tool_name='agent_hub',
cloudra/tools/registry.py:204           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:205           supervisor_access='hub',
cloudra/tools/registry.py:206       ),
cloudra/tools/registry.py:207       ToolRegistration(
cloudra/tools/registry.py:208           module='cloudra.tools.Managetool.get_subagent_report',
cloudra/tools/registry.py:209           class_name='GetSubAgentReport',
cloudra/tools/registry.py:210           tool_name='get_subagent_report',
cloudra/tools/registry.py:211           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:212       ),
cloudra/tools/registry.py:213       ToolRegistration(
cloudra/tools/registry.py:214           module='cloudra.tools.Managetool.schedule',
cloudra/tools/registry.py:215           class_name='Schedule',
cloudra/tools/registry.py:216           tool_name='schedule',
cloudra/tools/registry.py:217           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:218           invocation_direct_only=True,
cloudra/tools/registry.py:219       ),
cloudra/tools/registry.py:220       ToolRegistration(
cloudra/tools/registry.py:221           module='cloudra.tools.Managetool.subagent_launcher',
cloudra/tools/registry.py:222           class_name='SubAgentLauncher',
cloudra/tools/registry.py:223           tool_name='subagent_launcher',
cloudra/tools/registry.py:224           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:225       ),
cloudra/tools/registry.py:226       ToolRegistration(
cloudra/tools/registry.py:227           module='cloudra.tools.Managetool.subagent_swarm',
cloudra/tools/registry.py:228           class_name='SubAgentSwarm',
cloudra/tools/registry.py:229           tool_name='subagent_swarm',
cloudra/tools/registry.py:230       ),
cloudra/tools/registry.py:231       ToolRegistration('cloudra.tools.Plantool.goal', 'GoalSet'),
cloudra/tools/registry.py:232       ToolRegistration(
cloudra/tools/registry.py:233           module='cloudra.tools.Plantool.dag_tools.dag_view',
cloudra/tools/registry.py:234           class_name='DagView',
cloudra/tools/registry.py:235           tool_name='dag_view',
cloudra/tools/registry.py:236           dag_observe=False,
cloudra/tools/registry.py:237           supervisor_access='evidence_read',
cloudra/tools/registry.py:238       ),
cloudra/tools/registry.py:239       ToolRegistration(
cloudra/tools/registry.py:240           module='cloudra.tools.Plantool.dag_tools.dag_delivery_report',
cloudra/tools/registry.py:241           class_name='DagDeliveryReport',
cloudra/tools/registry.py:242           tool_name='dag_delivery_report',
cloudra/tools/registry.py:243           dag_observe=False,
cloudra/tools/registry.py:244           supervisor_access='evidence_read',
cloudra/tools/registry.py:245       ),
cloudra/tools/registry.py:246       ToolRegistration(
cloudra/tools/registry.py:247           module='cloudra.tools.Plantool.dag_tools.dag_add_species',
cloudra/tools/registry.py:248           class_name='DagAddSpecies',
cloudra/tools/registry.py:249           tool_name='dag_add_species',
```

```
cloudra/tools/registry.py:250           dag_observe=False,
cloudra/tools/registry.py:251       ),
cloudra/tools/registry.py:252       ToolRegistration(
cloudra/tools/registry.py:253           module='cloudra.tools.Plantool.dag_tools.dag_add_step',
cloudra/tools/registry.py:254           class_name='DagAddStep',
cloudra/tools/registry.py:255           tool_name='dag_add_step',
cloudra/tools/registry.py:256           dag_observe=False,
cloudra/tools/registry.py:257       ),
cloudra/tools/registry.py:258       ToolRegistration(
cloudra/tools/registry.py:259           module='cloudra.tools.Plantool.dag_tools.dag_update_delivery_element',
cloudra/tools/registry.py:260           class_name='DagUpdateDeliveryElement',
cloudra/tools/registry.py:261           tool_name='dag_update_delivery_element',
cloudra/tools/registry.py:262           dag_observe=False,
cloudra/tools/registry.py:263       ),
cloudra/tools/registry.py:264       ToolRegistration(
cloudra/tools/registry.py:265           module='cloudra.tools.Plantool.dag_tools.dag_remove_edge',
cloudra/tools/registry.py:266           class_name='DagRemoveEdge',
cloudra/tools/registry.py:267           tool_name='dag_remove_edge',
cloudra/tools/registry.py:268           dag_observe=False,
cloudra/tools/registry.py:269       ),
cloudra/tools/registry.py:270       ToolRegistration(
cloudra/tools/registry.py:271           module='cloudra.tools.Plantool.dag_tools.dag_remove_node',
cloudra/tools/registry.py:272           class_name='DagRemoveNode',
cloudra/tools/registry.py:273           tool_name='dag_remove_node',
cloudra/tools/registry.py:274           dag_observe=False,
cloudra/tools/registry.py:275           # 无 supervisor_access：删图是研究侧（Coordinator/子 agent）的修改
cloudra/tools/registry.py:276           # 权限；Supervisor 保留审查职责、不顺带获得直接删图能力。
cloudra/tools/registry.py:277       ),
cloudra/tools/registry.py:278       ToolRegistration(
cloudra/tools/registry.py:279           module='cloudra.tools.Plantool.dag_tools.dag_bind_evidence',
cloudra/tools/registry.py:280           class_name='DagBindEvidence',
cloudra/tools/registry.py:281           tool_name='dag_bind_evidence',
cloudra/tools/registry.py:282           dag_observe=False,
cloudra/tools/registry.py:283       ),
cloudra/tools/registry.py:284       ToolRegistration(
cloudra/tools/registry.py:285           module='cloudra.tools.Plantool.dag_tools.dag_propose_mechanism',
cloudra/tools/registry.py:286           class_name='DagProposeMechanism',
cloudra/tools/registry.py:287           tool_name='dag_propose_mechanism',
cloudra/tools/registry.py:288           dag_observe=False,
cloudra/tools/registry.py:289       ),
cloudra/tools/registry.py:290       ToolRegistration(
cloudra/tools/registry.py:291           module='cloudra.tools.Plantool.dag_tools.dag_submit_step_evidence',
cloudra/tools/registry.py:292           class_name='DagSubmitStepEvidence',
cloudra/tools/registry.py:293           tool_name='dag_submit_step_evidence',
cloudra/tools/registry.py:294           dag_observe=False,
cloudra/tools/registry.py:295       ),
cloudra/tools/registry.py:296       ToolRegistration(
cloudra/tools/registry.py:297           module='cloudra.tools.Plantool.dag_tools.dag_submit_species_evidence',
cloudra/tools/registry.py:298           class_name='DagSubmitSpeciesEvidence',
cloudra/tools/registry.py:299           tool_name='dag_submit_species_evidence',
cloudra/tools/registry.py:300           dag_observe=False,
cloudra/tools/registry.py:301       ),
```

```
cloudra/tools/registry.py:302       ToolRegistration(
cloudra/tools/registry.py:303           module='cloudra.tools.Plantool.dag_tools.dag_review_step',
cloudra/tools/registry.py:304           class_name='DagReviewStep',
cloudra/tools/registry.py:305           tool_name='dag_review_step',
cloudra/tools/registry.py:306           dag_observe=False,
cloudra/tools/registry.py:307           supervisor_access='review_decision',
cloudra/tools/registry.py:308       ),
cloudra/tools/registry.py:309       ToolRegistration(
cloudra/tools/registry.py:310           module='cloudra.tools.Plantool.dag_tools.dag_review_species',
cloudra/tools/registry.py:311           class_name='DagReviewSpecies',
cloudra/tools/registry.py:312           tool_name='dag_review_species',
cloudra/tools/registry.py:313           dag_observe=False,
cloudra/tools/registry.py:314           supervisor_access='review_decision',
cloudra/tools/registry.py:315       ),
cloudra/tools/registry.py:316       ToolRegistration(
cloudra/tools/registry.py:317           module='cloudra.tools.Plantool.dag_tools.dag_accept_mechanism',
cloudra/tools/registry.py:318           class_name='DagAcceptMechanism',
cloudra/tools/registry.py:319           tool_name='dag_accept_mechanism',
cloudra/tools/registry.py:320           dag_observe=False,
cloudra/tools/registry.py:321           supervisor_access='review_decision',
cloudra/tools/registry.py:322       ),
cloudra/tools/registry.py:323       ToolRegistration(
cloudra/tools/registry.py:324           module='cloudra.tools.Plantool.dag_tools.dag_review_call',
cloudra/tools/registry.py:325           class_name='DagReviewCall',
cloudra/tools/registry.py:326           tool_name='dag_review_call',
cloudra/tools/registry.py:327           dag_observe=False,
cloudra/tools/registry.py:328           supervisor_access='review_decision',
cloudra/tools/registry.py:329       ),
cloudra/tools/registry.py:330       ToolRegistration(
cloudra/tools/registry.py:331           module='cloudra.tools.Plantool.dag_tools.dag_review_evidence',
cloudra/tools/registry.py:332           class_name='DagReviewEvidence',
cloudra/tools/registry.py:333           tool_name='dag_review_evidence',
cloudra/tools/registry.py:334           dag_observe=False,
cloudra/tools/registry.py:335           supervisor_access='review_decision',
cloudra/tools/registry.py:336       ),
cloudra/tools/registry.py:337       ToolRegistration(
cloudra/tools/registry.py:338           module='cloudra.tools.Plantool.dag_tools.dag_view_evidence',
cloudra/tools/registry.py:339           class_name='DagViewEvidence',
cloudra/tools/registry.py:340           tool_name='dag_view_evidence',
cloudra/tools/registry.py:341           dag_observe=False,
cloudra/tools/registry.py:342           supervisor_access='evidence_read',
cloudra/tools/registry.py:343       ),
cloudra/tools/registry.py:344       ToolRegistration(
cloudra/tools/registry.py:345           module='cloudra.tools.Plantool.dag_tools.dag_set_coverage_contract',
cloudra/tools/registry.py:346           class_name='DagSetCoverageContract',
cloudra/tools/registry.py:347           tool_name='dag_set_coverage_contract',
cloudra/tools/registry.py:348           dag_observe=False,
cloudra/tools/registry.py:349           supervisor_access='review_decision',
cloudra/tools/registry.py:350       ),
cloudra/tools/registry.py:351       ToolRegistration(
cloudra/tools/registry.py:352           module='cloudra.tools.Plantool.dag_tools.dag_message_supervisor',
cloudra/tools/registry.py:353           class_name='DagMessageSupervisor',
```

```
cloudra/tools/registry.py:354            tool_name='dag_message_supervisor',
cloudra/tools/registry.py:355            dag_observe=False,
cloudra/tools/registry.py:356            # 无 supervisor_access：这是研究 Agent/Coordinator → Supervisor 的
cloudra/tools/registry.py:357            # 请求入口，工具本身硬拒 Supervisor 调用；督导的回复路径是
cloudra/tools/registry.py:358            # dag_review_* 裁决工具与运行结束报告。
cloudra/tools/registry.py:359        ),
cloudra/tools/registry.py:360        ToolRegistration('cloudra.tools.Plantool.todo', 'TodoWrite'),
cloudra/tools/registry.py:361        ToolRegistration('cloudra.tools.Plantool.todo_read', 'TodoRead'),
cloudra/tools/registry.py:362        ToolRegistration(
cloudra/tools/registry.py:363            module='cloudra.tools.Searchtool.read',
cloudra/tools/registry.py:364            class_name='Read',
cloudra/tools/registry.py:365            tool_name='read',
cloudra/tools/registry.py:366            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:367            supervisor_access='evidence_read',
cloudra/tools/registry.py:368        ),
cloudra/tools/registry.py:369        ToolRegistration(
cloudra/tools/registry.py:370            module='cloudra.tools.Searchtool.search',
cloudra/tools/registry.py:371            class_name='Search',
cloudra/tools/registry.py:372            tool_name='search',
cloudra/tools/registry.py:373            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:374        ),
cloudra/tools/registry.py:375        ToolRegistration(
cloudra/tools/registry.py:376            module='cloudra.tools.Skilltool.get_skill',
cloudra/tools/registry.py:377            class_name='GetSkill',
cloudra/tools/registry.py:378            tool_name='get_skill',
cloudra/tools/registry.py:379            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:380            dag_observe=False,
cloudra/tools/registry.py:381            supervisor_access='skill_read',
cloudra/tools/registry.py:382        ),
cloudra/tools/registry.py:383        ToolRegistration(
cloudra/tools/registry.py:384            module='cloudra.tools.Stoptool.stop_tool',
cloudra/tools/registry.py:385            class_name='StopTool',
cloudra/tools/registry.py:386            tool_name='stop',
cloudra/tools/registry.py:387            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:388            invocation_direct_only=True,
cloudra/tools/registry.py:389        ),
cloudra/tools/registry.py:390        ToolRegistration(
cloudra/tools/registry.py:391            module='cloudra.tools.Writetool.edit',
cloudra/tools/registry.py:392            class_name='Edit',
cloudra/tools/registry.py:393            tool_name='edit',
cloudra/tools/registry.py:394            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:395        ),
cloudra/tools/registry.py:396        ToolRegistration(
cloudra/tools/registry.py:397            module='cloudra.tools.Writetool.write',
cloudra/tools/registry.py:398            class_name='Write',
cloudra/tools/registry.py:399            tool_name='write',
cloudra/tools/registry.py:400            runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:401        ),
cloudra/tools/registry.py:402        ToolRegistration(
cloudra/tools/registry.py:403            module='cloudra.tools.Writetool.update_memory',
cloudra/tools/registry.py:404            class_name='UpdateMemory',
cloudra/tools/registry.py:405            tool_name='update_memory',
```

```
cloudra/tools/registry.py:406           runtime_profiles=frozenset({'desktop', 'android'}),
cloudra/tools/registry.py:407       ),
cloudra/tools/registry.py:408   )
cloudra/web_launcher.py:1   """Web frontend launcher used by the `cloudra` command.
cloudra/web_launcher.py:2
cloudra/web_launcher.py:3   Keeps Node/npm knowledge out of the CLI module.  The launcher works for source
cloudra/web_launcher.py:4   checkouts and editable installs; packaged wheels can point CLOUDRA_FRONTEND_DIR
cloudra/web_launcher.py:5   at an unpacked frontend directory.
cloudra/web_launcher.py:6   """
cloudra/web_launcher.py:7
cloudra/web_launcher.py:8   from __future__ import annotations
cloudra/web_launcher.py:9
cloudra/web_launcher.py:10  import logging
cloudra/web_launcher.py:11  import os
cloudra/web_launcher.py:12  import shutil
cloudra/web_launcher.py:13  import socket
cloudra/web_launcher.py:14  import subprocess
cloudra/web_launcher.py:15  import sys
cloudra/web_launcher.py:16  import time
cloudra/web_launcher.py:17  import urllib.request
cloudra/web_launcher.py:18  import webbrowser
cloudra/web_launcher.py:19  from pathlib import Path
cloudra/web_launcher.py:20
cloudra/web_launcher.py:21  from cloudra.services.daemon_supervisor import ensure_daemons
cloudra/web_launcher.py:22
cloudra/web_launcher.py:23  logger = logging.getLogger(__name__)
cloudra/web_launcher.py:24
cloudra/web_launcher.py:25
cloudra/web_launcher.py:26  def _frontend_dir() -> Path:
cloudra/web_launcher.py:27      override = os.environ.get("CLOUDRA_FRONTEND_DIR", "").strip()
cloudra/web_launcher.py:28      candidates = []
cloudra/web_launcher.py:29      if override:
cloudra/web_launcher.py:30          candidates.append(Path(override))
cloudra/web_launcher.py:31      # cloudra/web_launcher.py -> cloudra/.. == repository root (source checkout)
cloudra/web_launcher.py:32      candidates.append(Path(__file__).resolve().parents[1] / "frontend")
cloudra/web_launcher.py:33      candidates.append(Path.cwd() / "frontend")
cloudra/web_launcher.py:34      for candidate in candidates:
cloudra/web_launcher.py:35          if (candidate / "package.json").is_file():
cloudra/web_launcher.py:36              return candidate.resolve()
cloudra/web_launcher.py:37      raise RuntimeError(
cloudra/web_launcher.py:38          "frontend directory not found; set CLOUDRA_FRONTEND_DIR to the "
cloudra/web_launcher.py:39          "directory containing frontend/package.json"
cloudra/web_launcher.py:40      )
cloudra/web_launcher.py:41
cloudra/web_launcher.py:42
cloudra/web_launcher.py:43  def _check_node(frontend: Path) -> None:
cloudra/web_launcher.py:44      npm = shutil.which("npm")
cloudra/web_launcher.py:45      node = shutil.which("node")
cloudra/web_launcher.py:46      if npm is None or node is None:
cloudra/web_launcher.py:47          raise RuntimeError(
cloudra/web_launcher.py:48              "Node.js >= 18 and npm are required for the Cloudra web frontend. "
cloudra/web_launcher.py:49              f"Frontend found at {frontend}. Install from https://nodejs.org/"
```

```
cloudra/web_launcher.py:50          )
cloudra/web_launcher.py:51
cloudra/web_launcher.py:52
cloudra/web_launcher.py:53  def _npm_argv() -> list[str]:
cloudra/web_launcher.py:54      """npm as an invokable argv prefix.
cloudra/web_launcher.py:55
cloudra/web_launcher.py:56      On Windows npm is a ``npm.cmd`` batch file: ``shutil.which`` finds it (it
cloudra/web_launcher.py:57      honours PATHEXT) but ``CreateProcess`` cannot execute ``.cmd`` scripts
cloudra/web_launcher.py:58      directly — ``subprocess.run(["npm", ...])`` dies with ``WinError 2`` even
cloudra/web_launcher.py:59      though the pre-check passed.  Route through ``cmd /c`` with the fully
cloudra/web_launcher.py:60      resolved path; POSIX keeps the resolved executable directly.
cloudra/web_launcher.py:61      """
cloudra/web_launcher.py:62      npm = shutil.which("npm")
cloudra/web_launcher.py:63      if npm is None:
cloudra/web_launcher.py:64          raise RuntimeError(
cloudra/web_launcher.py:65              "npm not found on PATH; install Node.js >= 18 or run with --skip-build"
cloudra/web_launcher.py:66          )
cloudra/web_launcher.py:67      if os.name == "nt":
cloudra/web_launcher.py:68          return ["cmd", "/c", npm]
cloudra/web_launcher.py:69      return [npm]
cloudra/web_launcher.py:70
cloudra/web_launcher.py:71
cloudra/web_launcher.py:72  def _frontend_source_newer_than_dist(frontend: Path) -> bool:
cloudra/web_launcher.py:73      dist_index = frontend / "dist" / "index.html"
cloudra/web_launcher.py:74      if not dist_index.is_file():
cloudra/web_launcher.py:75          return True
cloudra/web_launcher.py:76      dist_mtime = dist_index.stat().st_mtime
cloudra/web_launcher.py:77      for root in (frontend / "src", frontend / "public"):
cloudra/web_launcher.py:78          if not root.is_dir():
cloudra/web_launcher.py:79              continue
cloudra/web_launcher.py:80          for path in root.rglob("*"):
cloudra/web_launcher.py:81              if path.is_file() and path.stat().st_mtime > dist_mtime:
cloudra/web_launcher.py:82                  return True
cloudra/web_launcher.py:83      for name in ("index.html", "vite.config.ts", "package.json"):
cloudra/web_launcher.py:84          path = frontend / name
cloudra/web_launcher.py:85          if path.is_file() and path.stat().st_mtime > dist_mtime:
cloudra/web_launcher.py:86              return True
cloudra/web_launcher.py:87      return False
cloudra/web_launcher.py:88
cloudra/web_launcher.py:89
cloudra/web_launcher.py:90  def _ensure_frontend(frontend: Path, *, rebuild: bool = False, skip_build: bool = False) ->
    None:
cloudra/web_launcher.py:91      _check_node(frontend)
cloudra/web_launcher.py:92      if not (frontend / "node_modules").is_dir():
cloudra/web_launcher.py:93          print("[cloudra] installing frontend dependencies...", flush=True)
cloudra/web_launcher.py:94          subprocess.run(_npm_argv() + ["install", "--no-audit", "--no-fund"], cwd=frontend,
    check=True)
cloudra/web_launcher.py:95      dist = frontend / "dist"
cloudra/web_launcher.py:96      if skip_build:
cloudra/web_launcher.py:97          if not (dist / "index.html").is_file():
cloudra/web_launcher.py:98              raise RuntimeError("--skip-build was requested but frontend/dist is missing")
cloudra/web_launcher.py:99          return
cloudra/web_launcher.py:100     stale = _frontend_source_newer_than_dist(frontend)
cloudra/web_launcher.py:101     if rebuild or not (dist / "index.html").is_file() or stale:
```

```
cloudra/web_launcher.py:102         if stale and not rebuild and (dist / "index.html").is_file():
cloudra/web_launcher.py:103             print("[cloudra] frontend source changed — rebuilding...", flush=True)
cloudra/web_launcher.py:104         else:
cloudra/web_launcher.py:105             print("[cloudra] building frontend...", flush=True)
cloudra/web_launcher.py:106         subprocess.run(_npm_argv() + ["run", "build"], cwd=frontend, check=True)
cloudra/web_launcher.py:107
cloudra/web_launcher.py:108
cloudra/web_launcher.py:109 def _port_is_bindable(preferred: int) -> bool:
cloudra/web_launcher.py:110     with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
cloudra/web_launcher.py:111         try:
cloudra/web_launcher.py:112             sock.bind(("127.0.0.1", preferred))
cloudra/web_launcher.py:113         except OSError:
cloudra/web_launcher.py:114             return False
cloudra/web_launcher.py:115         return True
cloudra/web_launcher.py:116
cloudra/web_launcher.py:117
cloudra/web_launcher.py:118 def _runtime_is_healthy(port: int) -> bool:
cloudra/web_launcher.py:119     try:
cloudra/web_launcher.py:120         with urllib.request.urlopen(
cloudra/web_launcher.py:121             f"http://127.0.0.1:{port}/api/v2/runtime/health", timeout=1.0
cloudra/web_launcher.py:122         ) as resp:
cloudra/web_launcher.py:123             return 200 <= resp.status < 300
cloudra/web_launcher.py:124     except Exception:
cloudra/web_launcher.py:125         return False
cloudra/web_launcher.py:126
cloudra/web_launcher.py:127
cloudra/web_launcher.py:128 def _runtime_serves_spa(port: int) -> bool:
cloudra/web_launcher.py:129     try:
cloudra/web_launcher.py:130         with urllib.request.urlopen(f"http://127.0.0.1:{port}/", timeout=1.0) as resp:
cloudra/web_launcher.py:131             body = resp.read(4096).decode("utf-8", "ignore")
cloudra/web_launcher.py:132             return 200 <= resp.status < 300 and '<div id="root"></div>' in body
cloudra/web_launcher.py:133     except Exception:
cloudra/web_launcher.py:134         return False
cloudra/web_launcher.py:135
cloudra/web_launcher.py:136
cloudra/web_launcher.py:137 def _find_api_port(preferred: int, env_name: str) -> tuple[int, bool]:
cloudra/web_launcher.py:138     """Return (port, reused_existing_runtime).
cloudra/web_launcher.py:139
cloudra/web_launcher.py:140     A runtime is only reused when it both serves the API and the built SPA on
cloudra/web_launcher.py:141     the same port; otherwise we start a fresh single-port runtime on a free
cloudra/web_launcher.py:142     port so the frontend never points at a stale or foreign server.
cloudra/web_launcher.py:143     """
cloudra/web_launcher.py:144     env_value = os.environ.get(env_name)
cloudra/web_launcher.py:145     if env_value:
cloudra/web_launcher.py:146         port = int(env_value)
cloudra/web_launcher.py:147         return port, _runtime_is_healthy(port) and _runtime_serves_spa(port)
cloudra/web_launcher.py:148     if _port_is_bindable(preferred):
cloudra/web_launcher.py:149         return preferred, False
cloudra/web_launcher.py:150     if _runtime_is_healthy(preferred) and _runtime_serves_spa(preferred):
cloudra/web_launcher.py:151         return preferred, True
cloudra/web_launcher.py:152     for port in range(preferred + 1, preferred + 100):
cloudra/web_launcher.py:153         if _port_is_bindable(port):
```

```
cloudra/web_launcher.py:154             return port, False
cloudra/web_launcher.py:155     raise RuntimeError(f"no free port near {preferred}; set {env_name} explicitly")
cloudra/web_launcher.py:156
cloudra/web_launcher.py:157
cloudra/web_launcher.py:158 def _wait_healthy(url: str, timeout: float = 30.0) -> None:
cloudra/web_launcher.py:159     deadline = time.monotonic() + timeout
cloudra/web_launcher.py:160     while time.monotonic() < deadline:
cloudra/web_launcher.py:161         try:
cloudra/web_launcher.py:162             with urllib.request.urlopen(url, timeout=1.0) as resp:
cloudra/web_launcher.py:163                 if 200 <= resp.status < 300:
cloudra/web_launcher.py:164                     return
cloudra/web_launcher.py:165         except Exception:
cloudra/web_launcher.py:166             pass
cloudra/web_launcher.py:167         time.sleep(0.25)
cloudra/web_launcher.py:168     raise RuntimeError(f"runtime did not become healthy at {url}")
cloudra/web_launcher.py:169
cloudra/web_launcher.py:170
cloudra/web_launcher.py:171 def _terminate_runtime(runtime: subprocess.Popen, *, grace: float = 20.0) -> None:
cloudra/web_launcher.py:172     """SIGTERM → 有界等待 → SIGKILL 升级 → reap。
cloudra/web_launcher.py:173
cloudra/web_launcher.py:174     SIGTERM 让 uvicorn 走优雅关停（lifespan 收尾：停准入 → 取消推理/收尾
cloudra/web_launcher.py:175     任务 → 关流/客户端 → 回收 JS host）；不许只发信号假定已回收（停止
cloudra/web_launcher.py:176     语义契约 6）。
cloudra/web_launcher.py:177     """
cloudra/web_launcher.py:178     if runtime.poll() is not None:
cloudra/web_launcher.py:179         return
cloudra/web_launcher.py:180     runtime.terminate()
cloudra/web_launcher.py:181     try:
cloudra/web_launcher.py:182         runtime.wait(timeout=grace)
cloudra/web_launcher.py:183         return
cloudra/web_launcher.py:184     except subprocess.TimeoutExpired:
cloudra/web_launcher.py:185         logger.warning("web runtime did not exit within %.0fs of SIGTERM; killing", grace)
cloudra/web_launcher.py:186     runtime.kill()
cloudra/web_launcher.py:187     try:
cloudra/web_launcher.py:188         runtime.wait(timeout=5)
cloudra/web_launcher.py:189     except subprocess.TimeoutExpired:
cloudra/web_launcher.py:190         logger.error("web runtime still alive after SIGKILL")
cloudra/web_launcher.py:191
cloudra/web_launcher.py:192
cloudra/web_launcher.py:193 def launch_browser(frontend: Path, *, rebuild: bool, skip_build: bool) -> int:
cloudra/web_launcher.py:194     api_port, reused_runtime = _find_api_port(8000, "CLOUDRA_API_PORT")
cloudra/web_launcher.py:195     _ensure_frontend(frontend, rebuild=rebuild, skip_build=skip_build)
cloudra/web_launcher.py:196     # 显式启动清除 `cloudra stop` 的持久关停 marker，否则 ensure 拒绝拉起。
cloudra/web_launcher.py:197     from cloudra.services.daemon_supervisor import clear_daemons_shutdown
cloudra/web_launcher.py:198
cloudra/web_launcher.py:199     clear_daemons_shutdown()
cloudra/web_launcher.py:200     ensure_daemons()
cloudra/web_launcher.py:201     from cloudra.infra.config import get_config
cloudra/web_launcher.py:202
cloudra/web_launcher.py:203     store_path = os.environ.get(
cloudra/web_launcher.py:204         "CLOUDRA_RUNTIME_STORE",
cloudra/web_launcher.py:205         str(Path(get_config().data_dir) / "runtime-v2.db"),
```

```
cloudra/web_launcher.py:206     )
cloudra/web_launcher.py:207     env = os.environ.copy()
cloudra/web_launcher.py:208     env["CLOUDRA_RUNTIME_STORE"] = store_path
cloudra/web_launcher.py:209     env["CLOUDRA_FRONTEND_SERVE_DIR"] = str(frontend / "dist")
cloudra/web_launcher.py:210     runtime = None
cloudra/web_launcher.py:211     if reused_runtime:
cloudra/web_launcher.py:212         print(f"[cloudra] reusing existing Cloudra runtime on port {api_port}", flush=True)
cloudra/web_launcher.py:213     else:
cloudra/web_launcher.py:214         print(f"[cloudra] starting web server on port {api_port}...", flush=True)
cloudra/web_launcher.py:215         runtime = subprocess.Popen(
cloudra/web_launcher.py:216             [
cloudra/web_launcher.py:217                 sys.executable,
cloudra/web_launcher.py:218                 "-m",
cloudra/web_launcher.py:219                 "uvicorn",
cloudra/web_launcher.py:220                 "cloudra.services.api_v2:create_default_v2_app",
cloudra/web_launcher.py:221                 "--factory",
cloudra/web_launcher.py:222                 "--host",
cloudra/web_launcher.py:223                 "127.0.0.1",
cloudra/web_launcher.py:224                 "--port",
cloudra/web_launcher.py:225                 str(api_port),
cloudra/web_launcher.py:226                 "--log-level",
cloudra/web_launcher.py:227                 "warning",
cloudra/web_launcher.py:228             ],
cloudra/web_launcher.py:229             env=env,
cloudra/web_launcher.py:230             # 固定子进程 cwd：`python -m` 会把 cwd 放到 sys.path 最前面，
cloudra/web_launcher.py:231             # cwd 下一个无 __init__.py 的 `cloudra` 目录会把已安装包遮蔽成
cloudra/web_launcher.py:232             # namespace 包（__version__ 缺失、子模块却能被 editable finder
cloudra/web_launcher.py:233             # 加载，报 "unknown location"）。frontend 目录已校验存在且不
cloudra/web_launcher.py:234             # 含 cloudra 子目录，是确定无遮蔽的工作目录。
cloudra/web_launcher.py:235             cwd=frontend,
cloudra/web_launcher.py:236         )
cloudra/web_launcher.py:237     try:
cloudra/web_launcher.py:238         _wait_healthy(f"http://127.0.0.1:{api_port}/api/v2/runtime/health")
cloudra/web_launcher.py:239         from cloudra.services.api_token import resolve_api_token
cloudra/web_launcher.py:240
cloudra/web_launcher.py:241         token = resolve_api_token()
cloudra/web_launcher.py:242         url = f"http://127.0.0.1:{api_port}"
cloudra/web_launcher.py:243         # 认证开启时把 token 放在 URL fragment 里交给浏览器：fragment 不进
cloudra/web_launcher.py:244         # HTTP 请求，前端启动时读入 sessionStorage 后从地址栏抹掉。
cloudra/web_launcher.py:245         shown_url = f"{url}/#token={token}" if token is not None else url
cloudra/web_launcher.py:246         print(f"[cloudra] web UI: {shown_url}", flush=True)
cloudra/web_launcher.py:247         if os.environ.get("CLOUDRA_NO_BROWSER", "").strip().lower() not in {"1", "true", "yes
"}:
cloudra/web_launcher.py:248             webbrowser.open(shown_url)
cloudra/web_launcher.py:249         if runtime is None:
cloudra/web_launcher.py:250             return 0
cloudra/web_launcher.py:251         return runtime.wait()
cloudra/web_launcher.py:252     except KeyboardInterrupt:
cloudra/web_launcher.py:253         return 0
cloudra/web_launcher.py:254     finally:
cloudra/web_launcher.py:255         if runtime is not None:
cloudra/web_launcher.py:256             _terminate_runtime(runtime)
```